

Real Time Group

Real-Time Embedded Concepts

A horizontal red arrow pointing to the right, spanning most of the width of the slide.

Preface

The course aims to teach the fundamentals of RTOS and Embedded Systems. Throughout the course we will discuss :

- Embedded Systems (CPU & Memory, Peripherals, builds, Bootloaders ..).
- Real-Time O.S. services and Linux Kernel objects.

This is essential for writing RTOS or Bare Board Embedded Applications using multithreading, IPC, interrupts, protocols such as RS232, IIC and more.

We hope you will get the most out of this course, please feel free to ask any questions you may have ☺.

* slides are updated for October 2023.

Contents (1/2)



**RT Embedded
Linux Solutions**

- What is an embedded system.
- What is a Real-Time system.
- Basics Of Developing For Embedded Systems.
- Embedded System Initialization .
- Real-Time Operating Systems.
- Tasks - Designing Threaded Programs.
- Tasks in FreeRTOS
- Inter Process Communication
- Mutexes & Semaphores.
- Message Queues.
- Interrupts and exceptions.
 - Software Interrupts – Signals
 - Hardware Interrupts

What Is an Embedded System?

What Is an Embedded System? (1)

An **Embedded system** is a **computer system designed to do** one or a few **dedicated and/or specific functions** – and is controlled by an **embedded processor**.

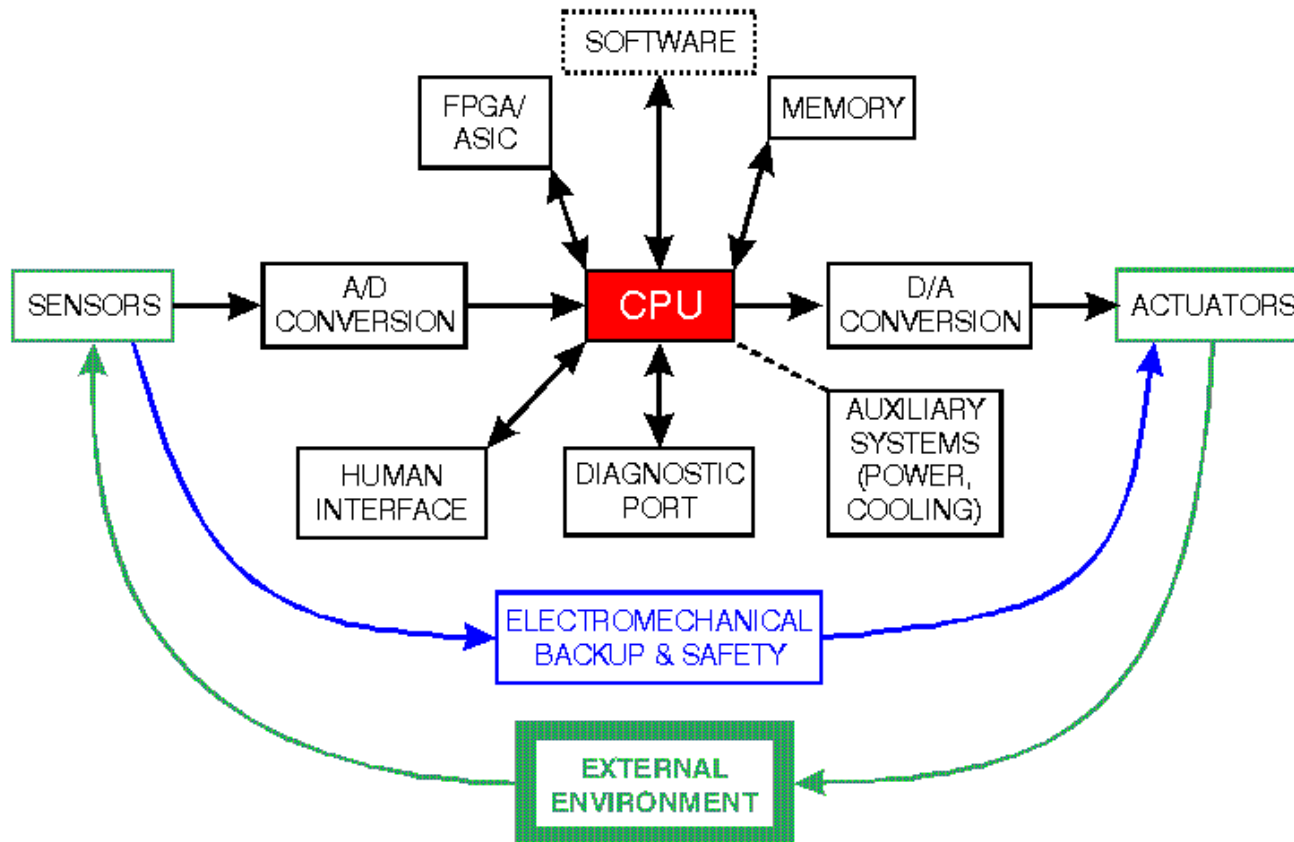
- The word **embedded** reflects the fact that **these systems are usually** an **integral part of a larger system**, known as the **embedding or hosting system**.
- Since Embedded **systems are dedicated to a specific task**, they are focused on (and optimized for) reduction in:
 - ✓ The size of the product
 - ✓ The cost of the product
 - ✓ The power consumption
- Embedded processors are **limited in functionality**.
- For example:
 - An **Embedded motion sensor** - can **only sense movement no more**
 - **Smart phones** on the other hand are **capable of running general purpose applications** such as **OFFICE or Adobe reader.. Email**

What Is an Embedded System? (2)

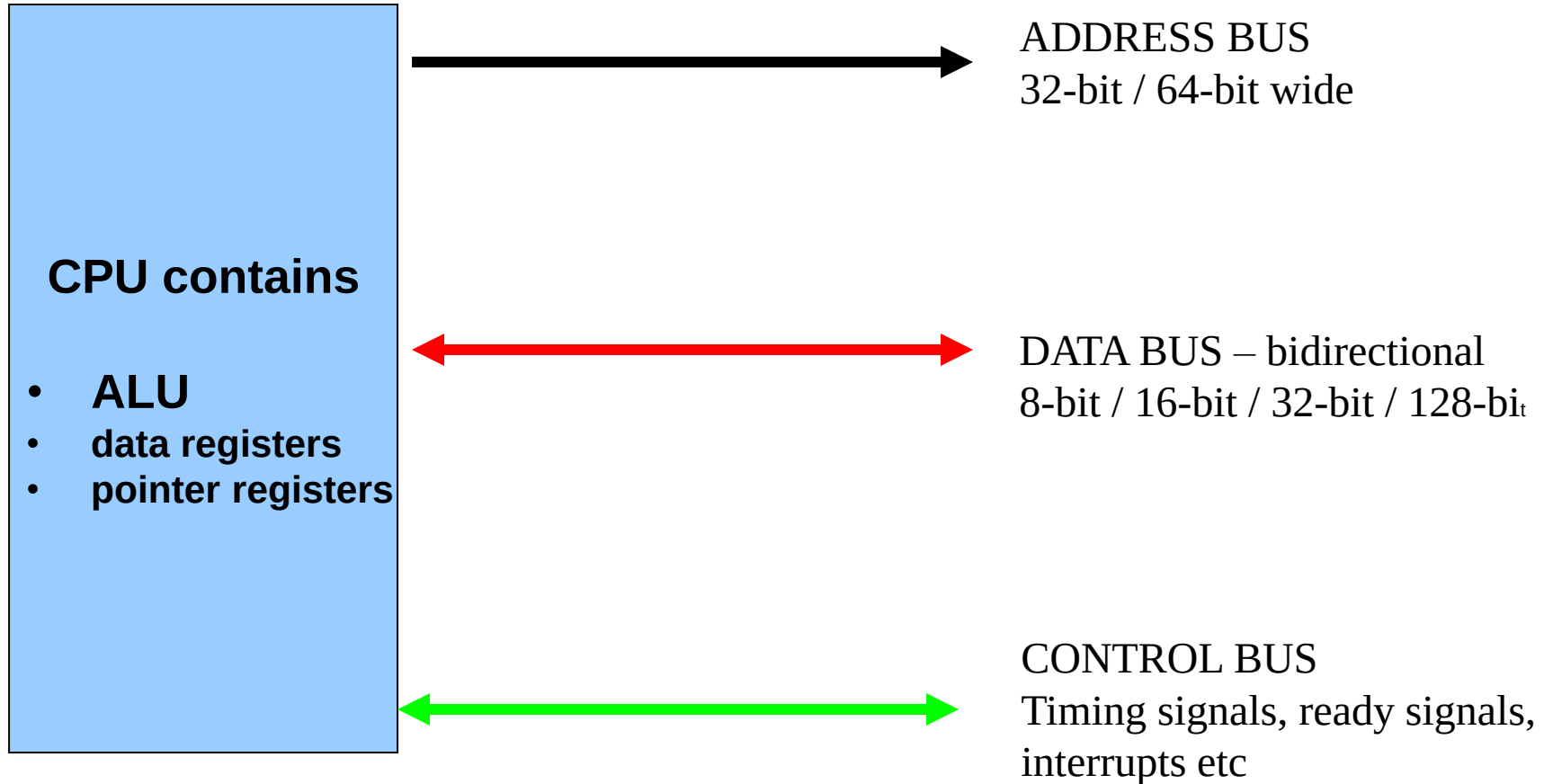
- Many Embedded **processors do not have fast CPU speeds**, because their **class of application doesn't need it**.
- Some **Embedded systems might use Micro-Controllers** not **Micro-Processors**
- In SOC (System On Chip) Systems CPU, RAM, ROM, I/O and timers are all on a single chip
 - Fixed amount of on-chip ROM, RAM, I/O ports
 - Applications in which cost, power and space are critical
 - single-purpose (control-oriented)
 - Low processing power \ Low power consumption
 - Instruction sets focus on control and bit-level operations
 - Processor might have **16-bit or 32 bit addressing architecture instead of 64-bit**, due to its limited memory storage capacity.

Embedded System Peripherals

- Designed for a specific purpose
- Based on some computational (software modifiable) element.
- Interacts with peripherals



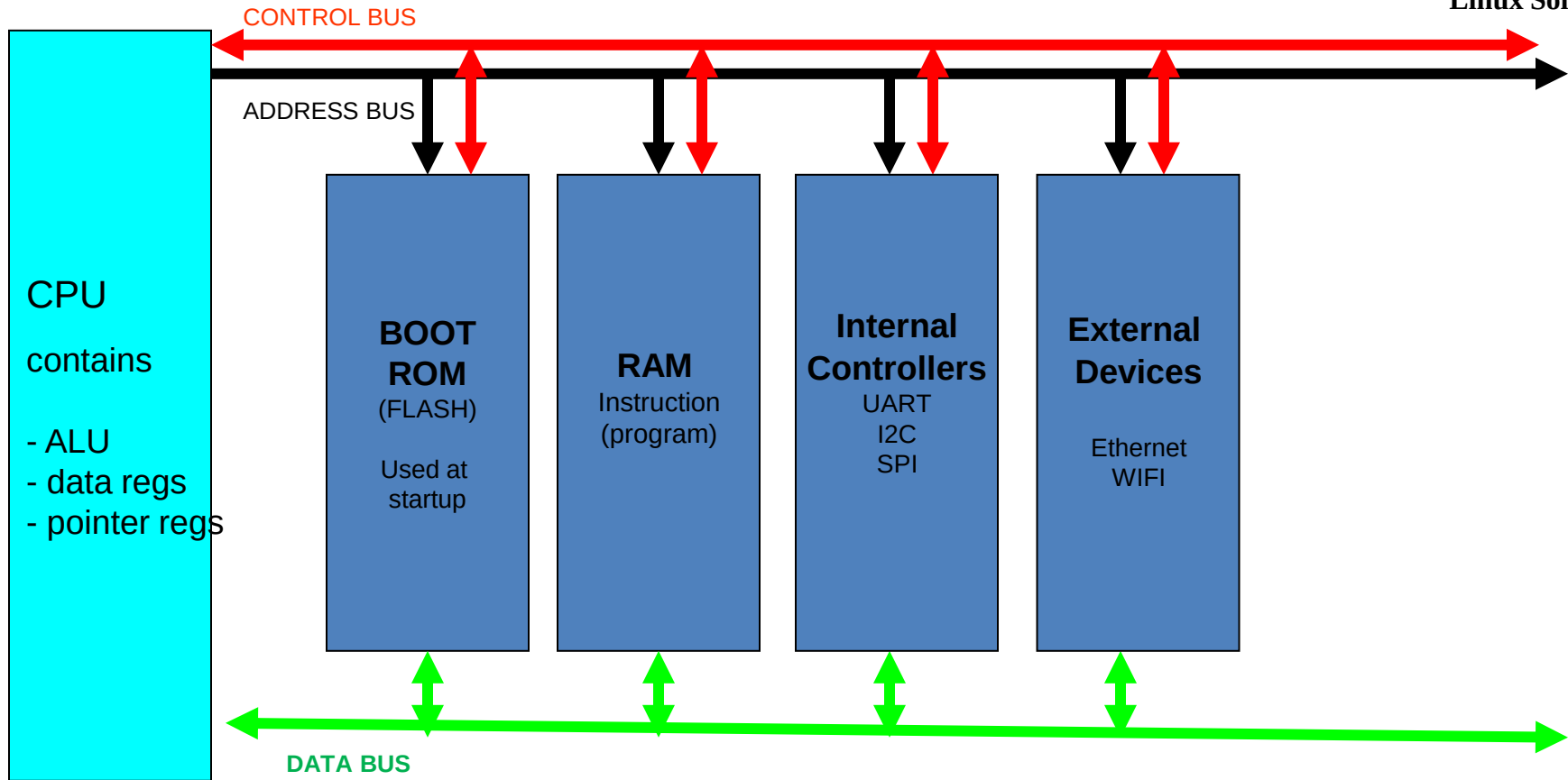
Microprocessor – Basic concept



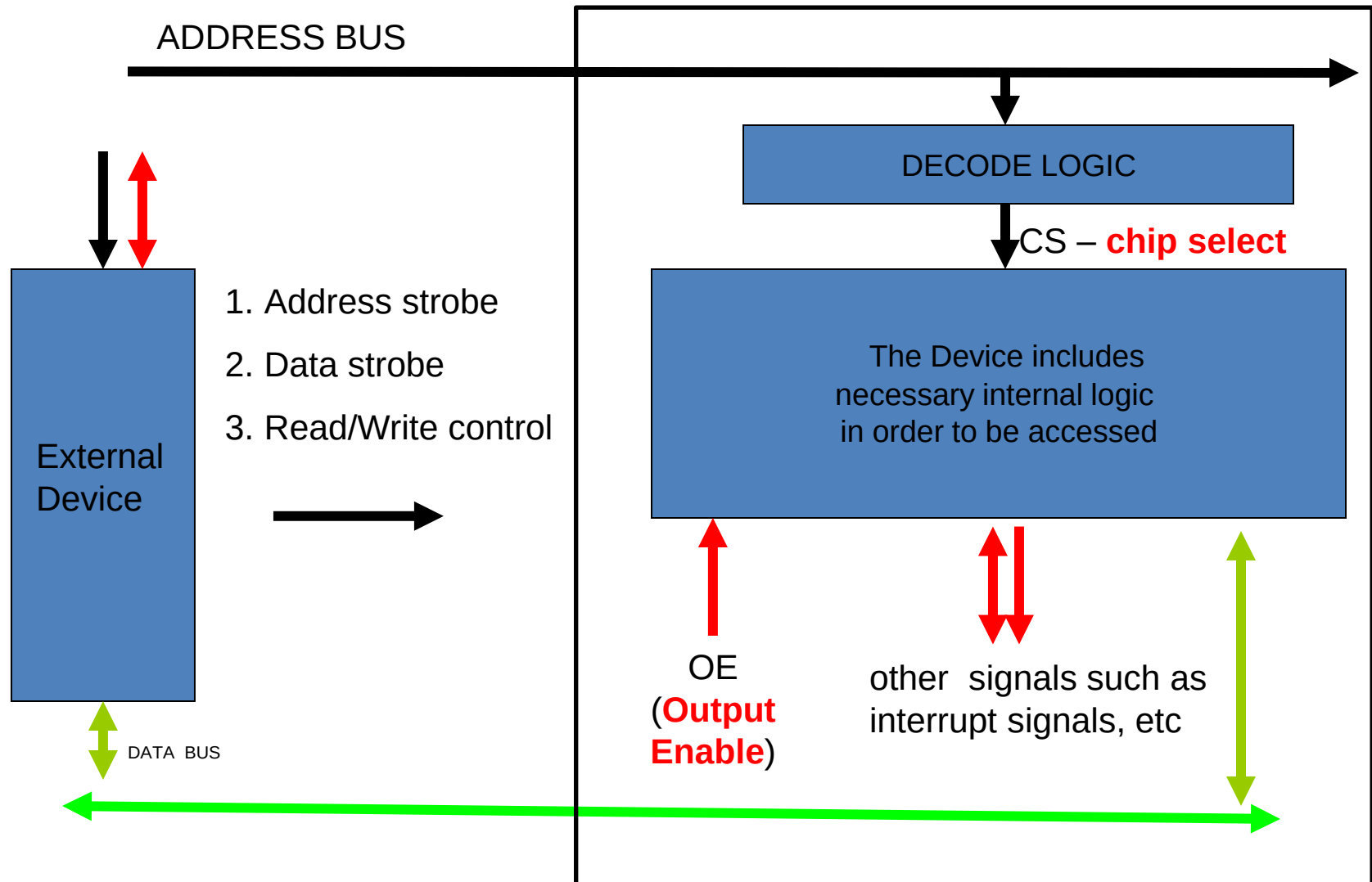
The Microprocessor, by-itself is completely useless –

- must have external peripherals to Interact with outside world

MicroPROCESSOR – Basic concept



Accessing external devices



Embedded Processor Types

- As mentioned previously, **embedded processors are specific CPU's suited (and built) for different class of applications**, for example:
 1. SOC (System On Chip) Processors.
 2. Networking \ Packet Processors
 3. Image processing Processors
- Nowadays in order to reduce development time, many SOM (System on Module) are used – Conga-Tech\ Phy-Tech \ TQ \ Verisite

System-on-a-chip processors

- **System-on-a-chip (SoC)** processors are especially attractive for embedded systems.
- The SoC processor is comprised of a **CPU core with built-in peripheral modules**, such as
 - Programmable general-purpose **timers**
 - **Programmable interrupt controller**,
 - **DMA controller**,
 - Serial protocol controllers (I2C \SPI \ UART...)
 - **Ethernet interfaces**.
- Such a self-contained design allows these embedded processors to be used to build a variety of **embedded applications without needing additional external peripheral devices**, reducing the overall cost and size of the final product.
- **Starting to look like general purpose CPU's**

SoC processor (S3C2440) Samsung (ARM A9)

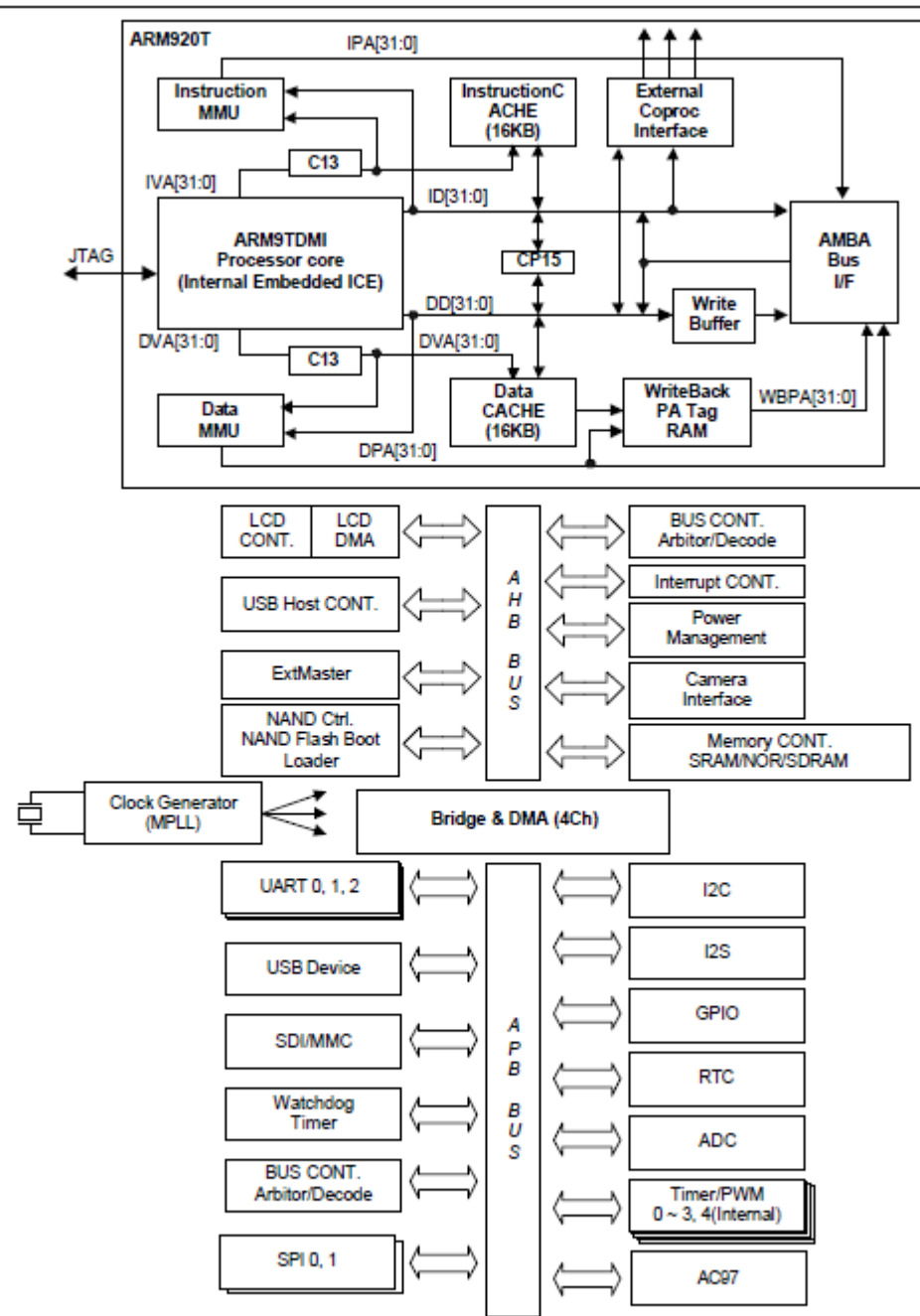
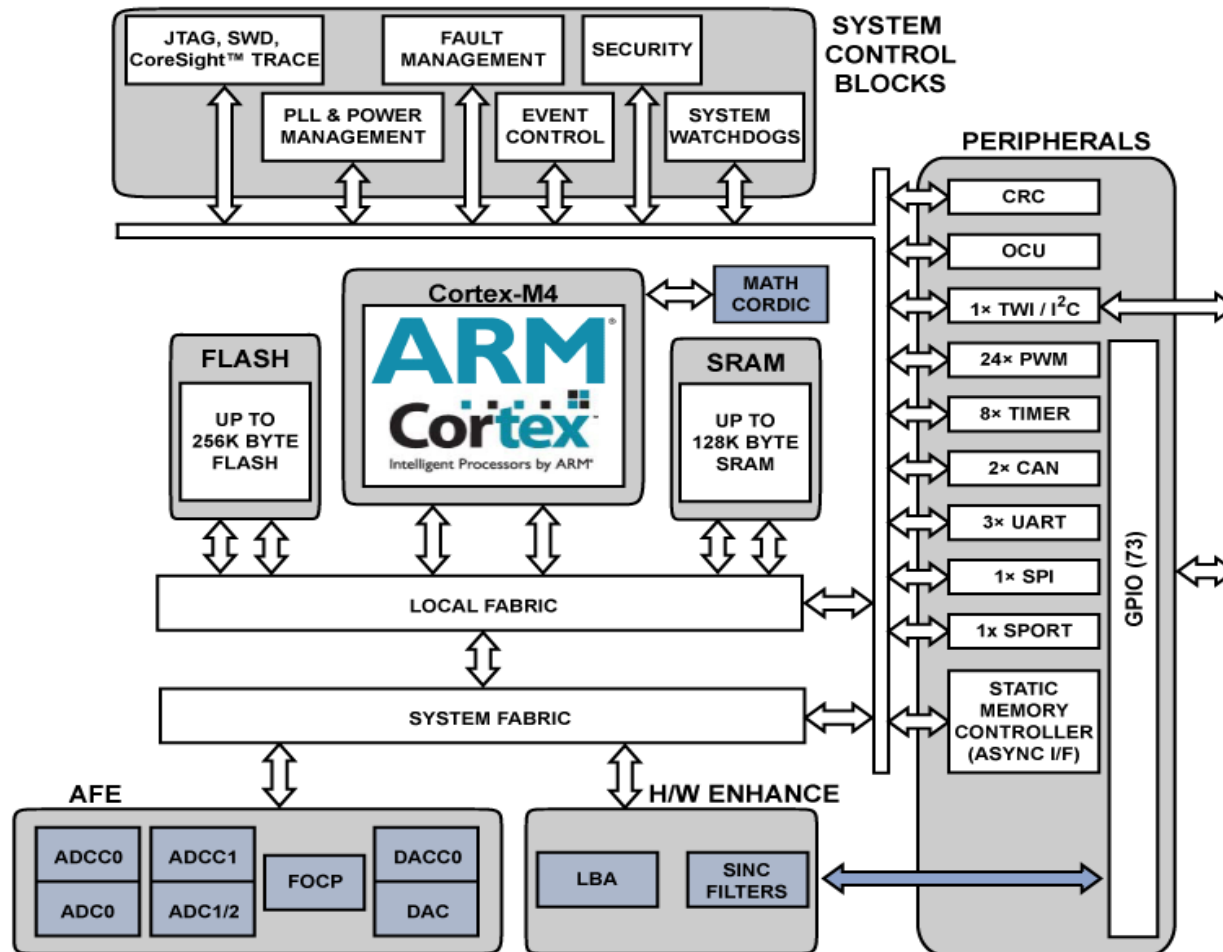


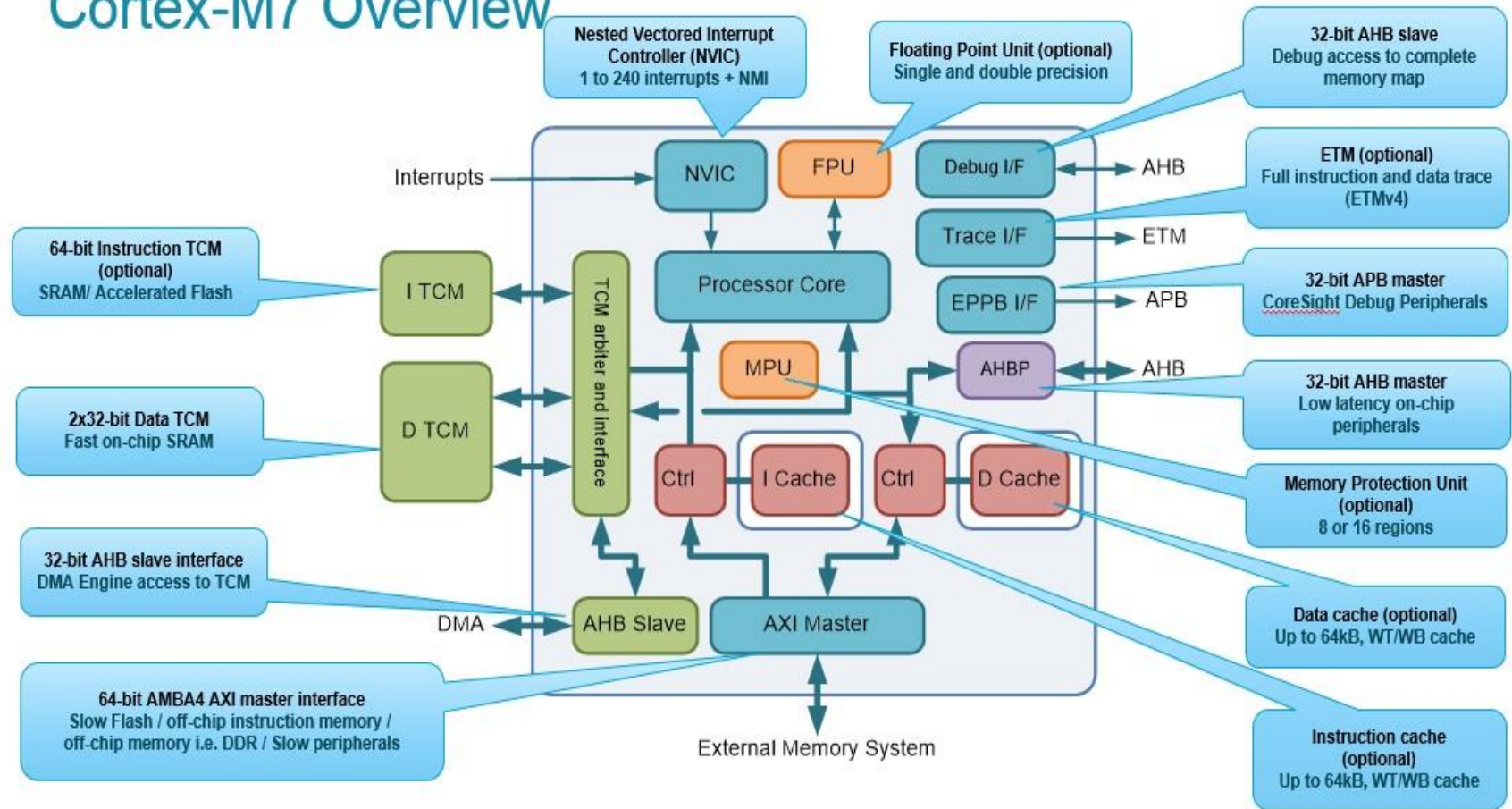
Figure 1-1. S3C2440A Block Diagram

ARM Cortex –M Series



ARM Cortex-M7

Cortex-M7 Overview



Network processors (1/2)

- A network processor is an integrated circuit which has a **feature set specifically targeted at the networking** application domain.
- Network Processors have specific features or architectures that are provided to **enhance and optimize packet processing** within networks, these include:
 - ✓ Pattern matching - finding specific patterns of bits or bytes within packets
 - ✓ Key lookup – a fast database lookup using a key (typically an field in a packet) to find a result (routing information).
 - ✓ Data bit-field manipulation - modifying data fields contained in the packet as it is being processed.
 - ✓ Queue management - as packets are received, processed and scheduled to be sent onwards, they are stored in queues.
 - ✓ Control processing - the micro operations of processing a packet are controlled at a macro level which involves communication and synchronization with other nodes in a system.
 - ✓ Quick allocation and re-circulation of packet buffers.

Network processors (2/2)

- Network processors use specific Hardware such as CAMs, frame buffers, CRC calculation blocks etc..
- Several architectural are commonly used:
 - ✓ Pipeline of processors - each stage of the pipeline consisting of a processor performing one of the functions listed above.
 - ✓ Parallel processing - multiple processors including multithreading.
 - ✓ Specialized microcoded engines to more efficiently accomplish the tasks at hand.
 - ✓ Multicore architectures are - used for higher layer (L4-L7) processing.
- Network processors are used in many different types of network equipment:
 1. Routers, switches.
 2. Firewalls .
 3. Intrusion detection \ prevention devices.
 4. Network monitoring systems.

Network processors (EZChip)

Task Optimized Processors (TOPs)

Four types of TOPs are featured, each tailored to perform its respective function:

<i>FUNCTION</i>	<i>TYPE OF TOP PROCESSOR</i>
Packet parse and classification	TOPparse
Lookup and classify	TOPsearch
Forwarding & QoS decisions	TOPresolve
Packet modify	TOPmodify

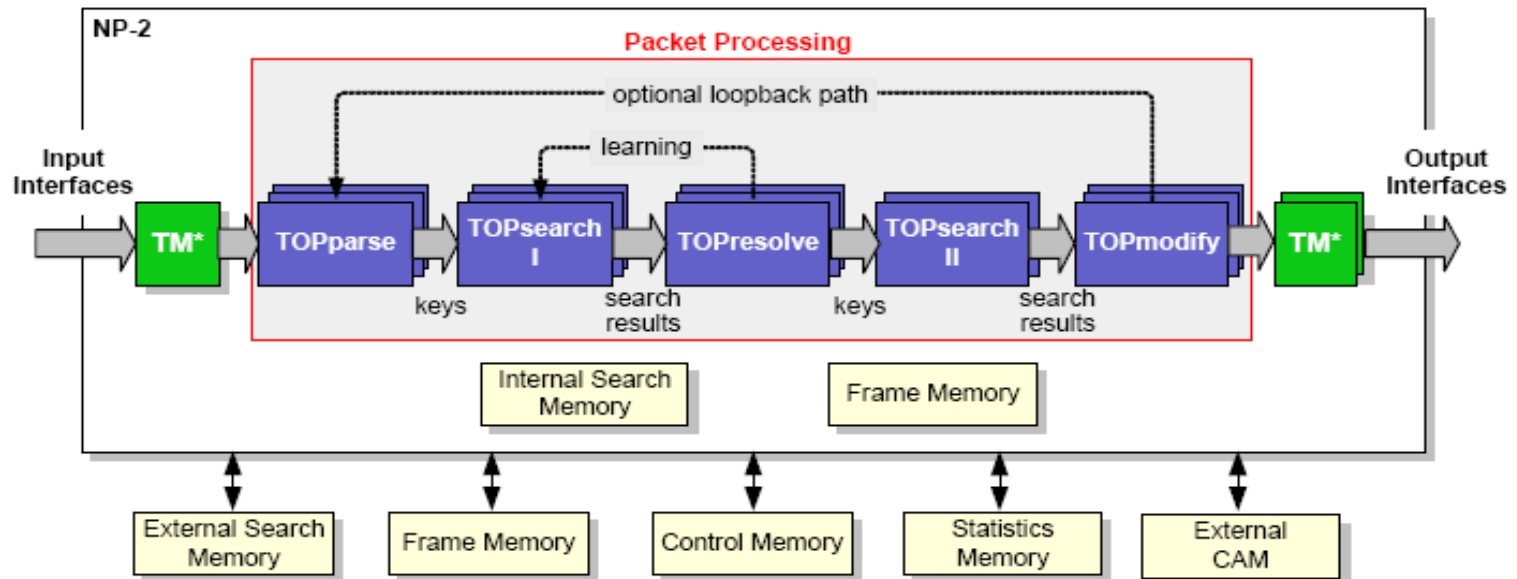
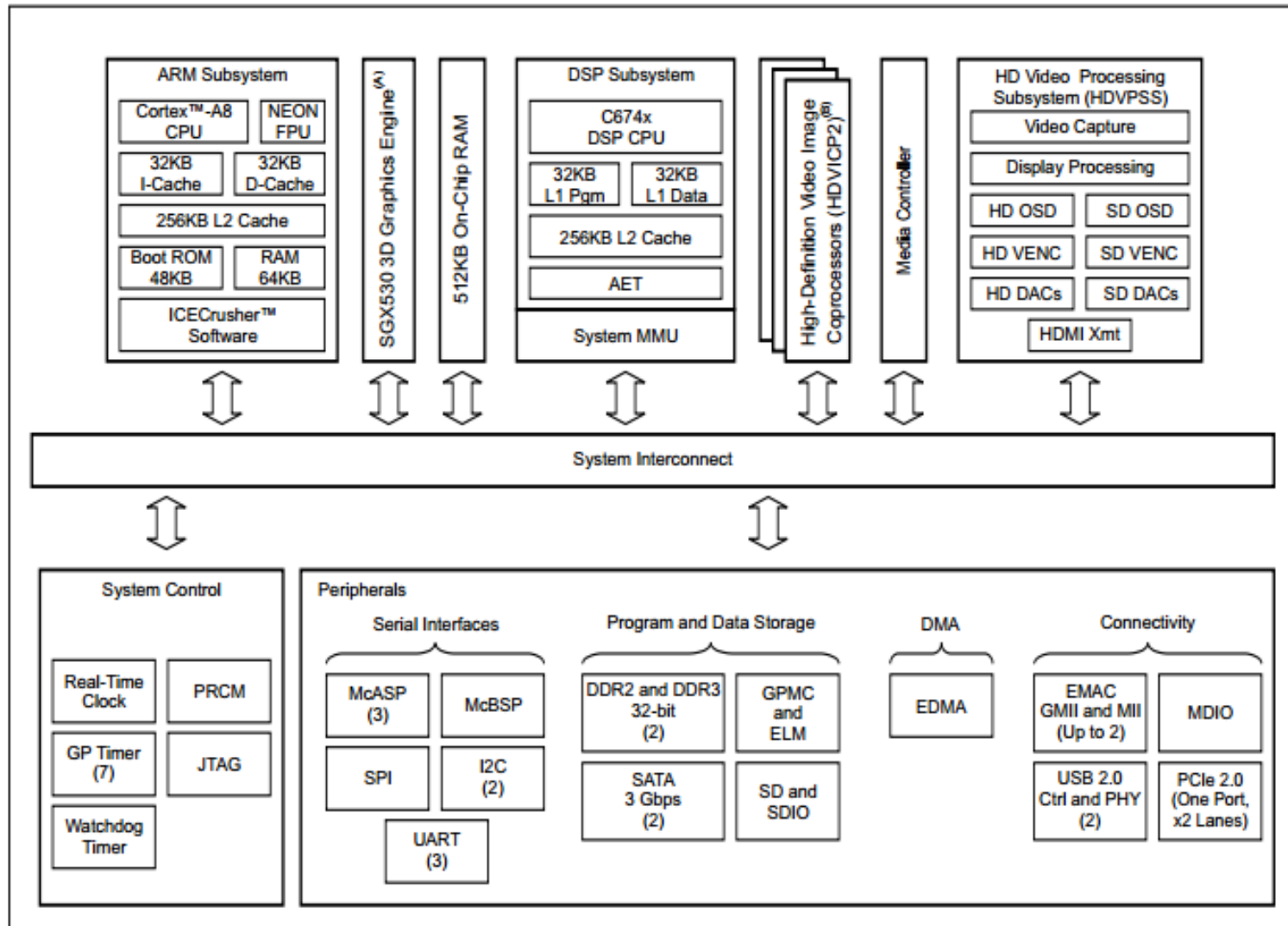


Image \ video processors

- These are processors that are primarily used in embedded video and vision applications.
- Many of the processors combine a DSP core and an General purpose CPU core into a single system on chip.
 - ARM - general-purpose processor – used for the control.
 - DSP - in charge of the media portion.
- For instance the Davinci
 - integrates several accelerators to offload commodity application
 - compression and decompression engine for HDVICP \ H.264 and MPEG-4.

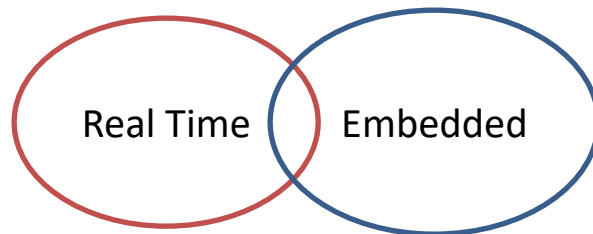
TI Davinci TMS320DM8

Functional Block Diagram



What Is a Real Time System?

- In the simplest form, Real-Time systems can be defined as those systems that **respond to external events in a timely fashion** (The response time must be **guaranteed**).
- Responding to external events includes :
 1. Recognizing when an event occurs.
 2. Performing the required processing.
 3. Outputting the necessary results **within a given time constraint**.
- not all embedded systems exhibit real-time behaviors, nor are all real-time systems embedded. However, the area in which they overlap creates the **combination of systems known as real-time embedded systems**.



Example: Real Time System (1/2)

- Imagine a **real-time weapons defense** system whose role is to protect a naval destroyer by shooting down incoming missiles.
 - The idea is to shoot down an incoming missile with bullets before it reaches the ship.
 - The weapons system is comprised of a
 1. **Radar System** - Scans and searches for potential targets. Coordinates of a potential target are sent to the C&D system periodically.
 2. **Command-and-Decision (C&D) system**- Determines the threat level, If a threat is imminent, the C&D system calculate the speed and flight path or trajectory, as well as estimate the impact location, an area (a box) around the flight path is made.

Example: Real Time System (2/2)

3. **Weapons firing control system** – The firing control system is guided to fire continuously within the moving area or box until the target is destroyed.
- The C&D system must use a **fast, accurate and deterministic** computation **algorithm**:
 - ✓ if the C&D system **took too long in computing** the flight path, the **missile would reach its target**.
 - ✓ if the **computation was inaccurate**, The **missile would still reach its target**.

Characteristics of Real-Time Systems



RT Embedded
Linux Solutions

- From the last example we can gather two essential characteristics of real-time systems:
 1. Real-Time systems must produce **correct computational results**, called logical or **functional correctness**,
 2. These computations must conclude within **a predefined period**, called **timing correctness**.
- Real-time systems are defined as those systems in which the overall correctness of the system depends on both the functional correctness and the timing correctness.
- The **timing correctness is at least as important as the functional correctness**.

Hard and Soft RT Systems (1/2)

- As mentioned computation must be completed before the deadline.
- Real-time systems have timing constraints and are deadline-driven, they can be classified as either:
 - ✓ **Hard Real-Time Systems**
 - ✓ **Soft Real-Time Systems**

The difference lies in:

1. degree of tolerance of missed deadlines,
2. usefulness of computed results after missed deadlines,
3. severity of the penalty incurred for failing to meet deadlines.

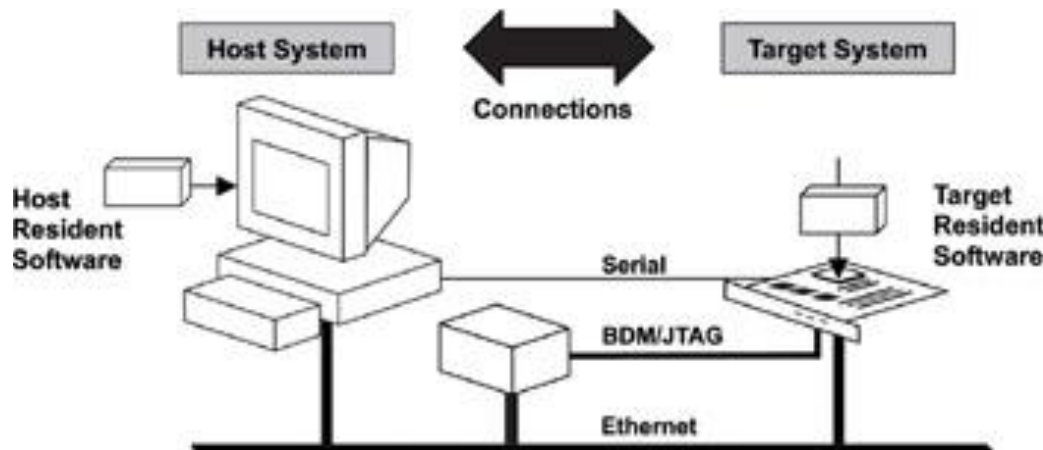
Hard and Soft RT Systems (2/2)

- **Hard Real-Time System** – is real-time systems that must meet its deadlines with a **near-zero degree of flexibility**.
 - ✓ The deadlines must be met, or **catastrophes occur**.
 - ✓ The cost of such catastrophe is extremely high and can involve human lives.
 - ✓ The computation results obtained after the deadline have a zero-level of usefulness.
- **Soft Real-Time Systems** - is a real-time system that must meet its deadlines but with a degree of flexibility.
 - The deadlines can **contain varying levels of tolerance**.
 - A missed deadline does not result in system failure, but costs can rise in proportion to the delay, depending on the application.
 - The computation results obtained after the deadline can be used to optimize the system.

Basics Of Embedded Systems Development

Cross-Platform Development (1)

- This is a method of software development, for both
 - ✓ Embedded **Bare-Boared \ Bare Metal Applications**
 - ✓ Embedded O.S. based **application software (Embedded Linux)**.
- **Software for an embedded system is developed on one platform** (usually the pc. – Linux \ Windows workstation) for use on another (the embedded) platform.



Cross-Platform Development (2)

- The platform is the combination of
 - ✓ hardware (such as a particular type of processor) .
 - ✓ operating system.
 - ✓ software development tools used for further development.
- **Host System** - is the system on which the embedded software is developed.
- **Target system** - is the embedded system under development.
- The main software tool that makes cross-platform development possible is a **cross compiler**.
 - ✓ **cross compiler** - a compiler that runs on one type of processor architecture but produces object code (**opcode**) for a different type of processor architecture.

Cross-Platform Development (3)

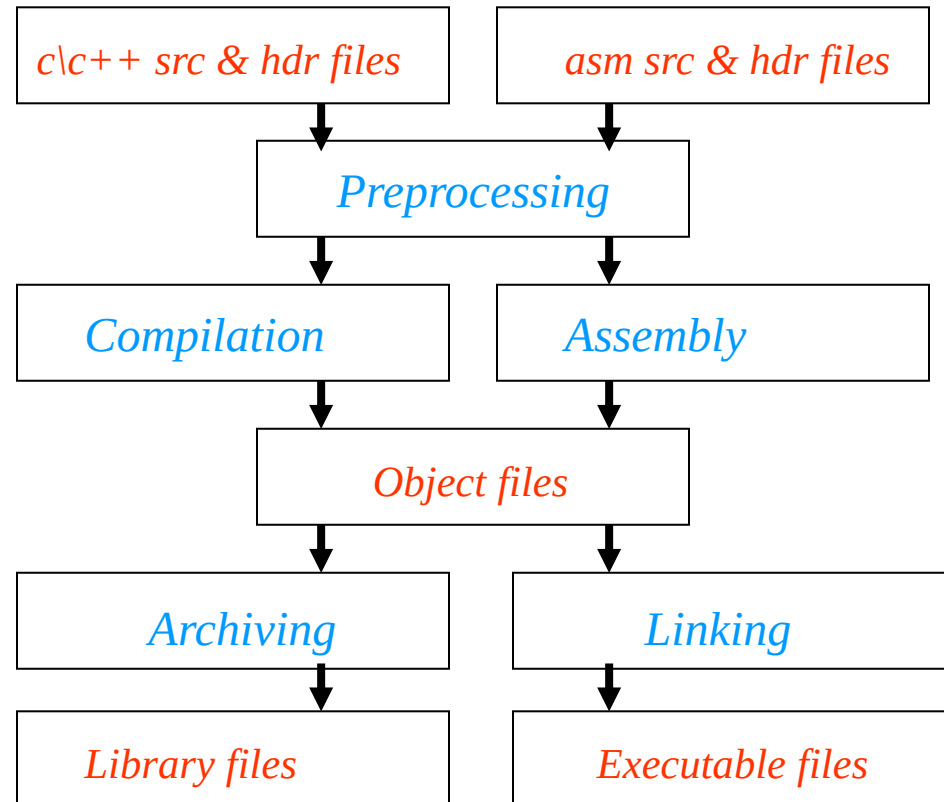
- For example, the ARM Cross compiler:
 - The development is done on a Linux operating system IA-32 \ AMD.
 - The cross compiler makes it possible to run the developed code on different architectures.
 - The compiler can produce object code for numerous processor types, such as Motorola s 68000, MIPS, and ARM.

Note:

Nowadays some architectures \ EVB supply their own tool chain and enable the development to be done on the Target itself (i.e. PCDuino\ raspberry-pi)

Compiling and Linking Process (1)

- The developer writes the *program in the C/C++/Asm source files* and header files.
- The **Makefile** is used by the make utility to facilitate an environment that can easily *track the file modifications and invoke the compiler and the assembler to rebuild* the source files when necessary.
- Produced object files that contain both *machine binary code and program data* are used by the archive utility to form a library.



The Compiling Process

- The **compiler translates programs written in some human-readable language into an equivalent set of op-codes for a particular processor.**
- **Each processor has its own unique machine language**, so you need to choose a compiler that is capable of producing programs for your specific target processor.
- A compiler such as this-that **runs on one computer platform and produces code for another-is called a cross-compiler.**
- The use of a cross-compiler is one of the defining features of embedded software development.
- The **output of the cross-compiler will be an object file.**
 - ✓ This is a specially formatted **binary file that contains the set of instructions and data** resulting from the language translation process.
 - ✓ Although parts of this file contain executable code, **the object file is not intended to be executed directly.**

Using Cross-Compilers

- To convert C source code to a binary object file:
arm-elf-gcc -c -O2 -g -mcpu=arm7tdmi filename.c -o filename.o
- To convert multiple binary object files into an executable file (the general case):
arm-elf-ld filename1.o filename2.o ... -o filename.elf
- To convert the assembly language code into re-locatable binary object output.
arm-elf-as filename1.s ... -o filename.O

Tool Chain Utilities



RT Embedded
Linux Solutions

Binary utilities:

- **arm-elf-ar** (ar) - create, modify, and extract from archives (Libraries)
- **arm-elf-nm** (nm) - list symbols from object files
- **arm-elf-objcopy** (objcopy) - copy and translate object files to different file types
- **arm-elf-objdump** (objdump) - displays information about one or more object files.
- **arm-elf-ranlib** (ranlib) - generates an index to the contents of an archive and stores it in the archive. The index lists each symbol defined by a member of an archive that is a relocatable object file.
- **arm-elf-readelf** (readelf) - displays information about one or more ELF format object files.
- **arm-elf-size** (size) - lists the section sizes---and the total size---for each of the object or archive files *objfile* in its argument list
- **arm-elf-strings** (strings) - print the strings of printable characters in files
- **arm-elf-strip** – (strip) - discards all symbols from object files *objfile*. The list of object files may include archives.
- **arm-elf-gdb** - Debugger

Example of a makefile.



**RT Embedded
Linux Solutions**

```
AL=arm-elf-ld
AS=arm-elf-as
NM=arm-elf-nm
OB=arm-elf-objcopy
CC=arm-elf-gcc
RT-ED-ROOT=../../
BSP_DIR=BSP
BOARD_BSP_DIR=mini2440
TARGET_DIR=mini2440
LIBPATH=$(ProgramFiles)/GNUARM/lib/gcc/arm-elf/4.1.1
```

```
default: img.bin
```

```
clean:
```

```
    rm -f *.o *.elf *.map
```

```
NM:
```

```
    $(NM) -n img.elf
```

```
img.elf: $(RT-ED-ROOT)/$(BSP_DIR)/BOARD/$(BOARD_BSP_DIR)/init.o main.o
    $(AL) -T img.lds -lgcc -L $(RT-ED-ROOT)/$(BSP_DIR)/BOARD/$(BOARD_BSP_DIR) -L
    $(LIBPATH) -L $(RT-ED-ROOT)/COMMON/SRC -nostdlib -Map img.map -o img.elf
```

```
main.o: $(RT-ED-ROOT)/COMMON/SRC/main.c
    $(CC) -g -c $(RT-ED-ROOT)/COMMON/SRC/main.c -march=armv4 -o main.o
```

```
img.bin:img.elf
    $(OB) img.elf -O binary img.bin
```

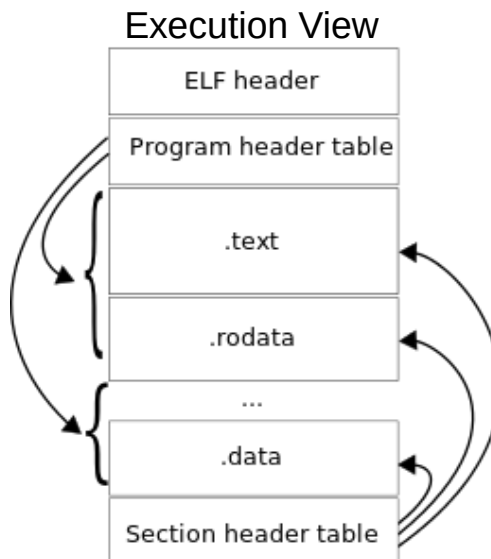
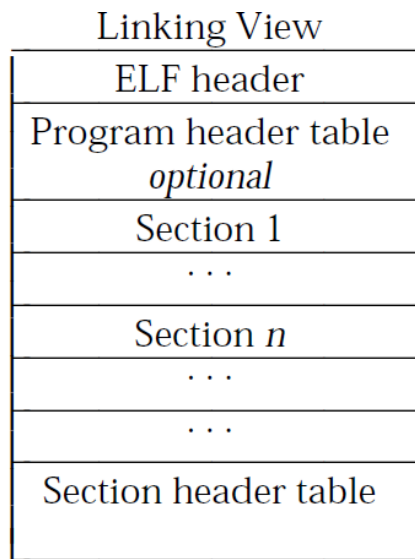
The Compiling Process

- The contents of an object file can be thought of as a very large, flexible data structure.
- The structure of the file is usually defined by a standard format like the following:
 - Common Object File Format (COFF)
 - Extended Linker Format (ELF).
- So What is a library ?
Instead of linking all object files and creating a run-able file as the above,
A library can be formed by concatenating a collection of object files using the archive utility.

The Linking Process

- The **object files** by themselves are **individually incomplete** some of the **internal variable and function references have not yet been resolved**.
- The **job of the linker** is to **combine these object files** and to **resolve all of the unresolved symbols**.
- The object file consists of three segments:
 - ✓ **Data** - **global and static variables** used by the program that are initialized.
 - ✓ **BSS** - **global and static variables** that are initialized to zero or do not have explicit initialization in source code .
 - ✓ **Text** – Code.
- The **linker merges** the text, data and bss sections from each input file into **one re-locatable entity with combined text, data and bss segments**.
- This image can be either:
 - ✓ Loaded by the OS loader which parses its contents a binary form at run-time.
 - ✓ Converted by objcopy utility into a binary format file in which is directly copied to the DRAM for execution.

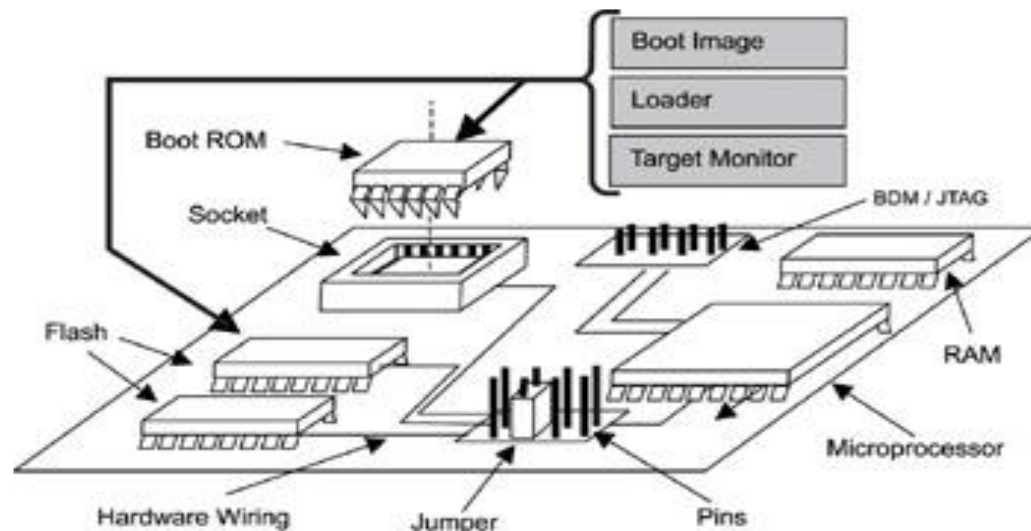
ELF file format (Executable and Linkable Format)



- The program header shows the segments used at run-time
- the section header lists the set of sections of the binary.

- An **ELF header** resides at the beginning and holds a “road map” describing the file’s organization.
- A **Program header table**, if present, tells the system **how to create a process image**. Files used to build a process image (execute a program) must have a program header table; re-locatable files do not need one.
- **Sections hold the bulk of object file information** for the linking view: **instructions, data, symbol table**, relocation information, and so on.
- A **Section header table** contains information describing the file’s sections. Every section has an entry in the table; each entry gives information such as **the section name, the section size**, etc.

Transferring the image to the target (1/2)



An executable **image built** for a target embedded system can be transferred from the host development system onto the target in different ways.

Transferring the image to the target (2/2)

1. Programming the entire image into the ROM device (typically EEPROM or FLASH memory) – hardly done nowadays .
2. Downloading the image over either USB interface or network connection (for small images even serial connection will suffice)

This process requires the presence of:

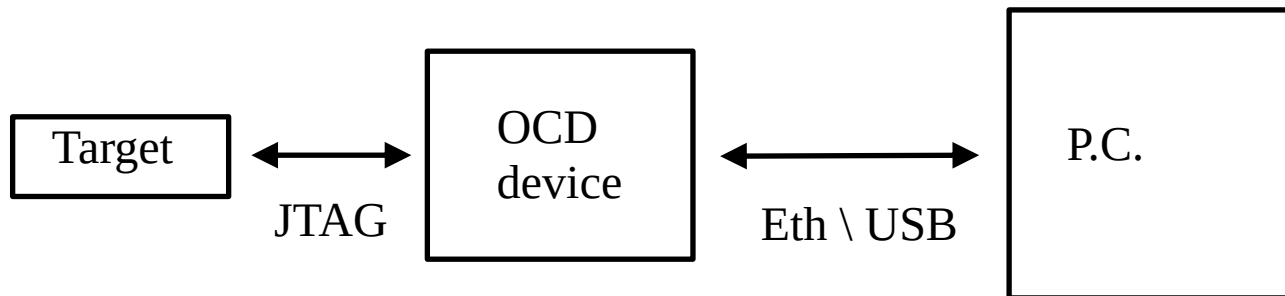
- a data transfer utility program on the host system,
 - a target loader, an embedded monitor, or a target debug agent on the target system.
3. Downloading the image through either a JTAG or BDM interface
 - The USB interface and the network connection are used for downloading the image to the target system's RAM memory. .
 - JTAG tools may be used as well and can be much faster
 - since they transfer the data directly without any use of target software.

On-Chip Debugging (1/2)

- The silicon vendors have recognized the need for built-in microprocessor debugging, called on-chip debugging (OCD).
- JTAG is the most used OCD solution that allows direct access and control over the microprocessor and system resources.
 - ✓ No need for software debug agents on the target
 - ✓ Or expensive in-circuit emulators.
- The embedded processor with OCD capability provides an external interface. Using this interface The developer can:
 - Download code,
 - Read or write processor registers,
 - Modify system memory, even burn FLASH
 - Command the processor to execute one instruction and halt (single-step debugging).

On-Chip Debugging (2/2)

- **JTAG** - **Joint Test Action Group**, which was founded by electronics manufacturers to develop a new and cost-effective test solution. The result, produced by the JTAG consortium, is sanctioned by the IEEE1149.1 standard.
- Many Microprocessors use JTAG as OCD interface.
 - ✓ This interface was **originally designed for boundary scans** and uses at least 5 lines (describes next slide).
- An OCD solution is comprised of both hardware and software.
 - ✓ The host side of the connection can be the USB port, or the network interface.
 - ✓ The **OCD-aware host debugger displays system state information**, such as the **contents of the processor registers**, the **system memory dump**, and **the current executing instruction**.



On-Chip Debugging (3/2)

Pin Name	Type	Description
TMS	Input	Test Mode Select- selects the next state in the TAP state machine
TCS	Input	Test Clock- This allows shifting of the data in, on the TMS and TDI pins. It is a positive edge-triggered clock with the TMS and TCK signals that define the internal state of the device.
TDI	Input	Test Data In- the serial data input for the shift register.
TDO	Output	Test Data Output- This is the serial data output from the shift register. Data is shifted out of the device on the negative edge of the TCK signal
nTRST	Input	Test Reset- The nRTST pin can be used to reset the test logic within the Embedded ICE logic.
RTCK	Output	Returned Test Clock- Extra signal added to the JTAG port. Required for designs based on ARM7TDMI-S processor core. Multi-ICE (Development system from ARM) uses this signal to maintain synchronization with targets having slow or widely varying clock frequency.

Embedded System Initialization

Storage types on target device

- Code for embedded systems (such as the real-time embedded operating system, the system software, and the application software) is stored on “ROM” devices (FLASH Memory).
- The embedded system **should be able to be upgraded dynamically** over a network (without the need for chip replacement).
 - The FLASH device memory enables this feature.
- Embedded systems use two different types of Memory devices:
 - ✓ ROM Devices
 - ✓ RAM Devices

Read Only Memory (ROM) (1)

- With non-volatile content and without the need for an external power source.
 - **Mask Programmed ROM:**
 - ✓ The memory content is programmed during the manufacturing process.
 - ✓ Once programmed, the content cannot be changed (It cannot be reprogrammed).
 - **Field Programmable ROM (PROM):**
 - ✓ The memory content can be custom-programmed one time.
 - ✓ The memory content cannot change once programmed
 - **Erasable Programmable ROM (EPROM):**
 - ✓ An EPROM device can be custom-programmed, erased, and reprogrammed as often as required.
 - ✓ Its lifetime is hundreds or even thousands of times.
 - ✓ The memory content is non-volatile once programmed.

Read Only Memory (ROM) (2)

- **Erasable Programmable ROM (Continued):**

- ✓ EPROM devices are erased by exposure to ultraviolet (UV) light.
- ✓ An EPROM device must be removed from its housing unit first.
- ✓ It is then reprogrammed using a special hardware device called an EPROM programmer.

- **Electrically Erasable Programmable ROM (EEPROM or E2PROM) :**

- ✓ EEPROM devices are erased electrically.
- ✓ One important difference between an EPROM and an EEPROM device is that with the EEPROM device, memory content of a single byte can be selectively erased and reprogrammed.
- ✓ EEPROM devices allow incremental changes.
- ✓ Another difference is the EEPROM can be reprogrammed without a special programmer and can stay in the device while being reprogrammed.
- ✓ Programming an EEPROM device is a slow process.

Flash Memory

- Allows block-level (e.g., 512-byte) programmability.
- Much faster than EEPROM, memory content of a single byte can be selectively erased and reprogrammed.
- Incremental changes can be made.
- The Flash can be reprogrammed and stay in the device while being reprogrammed.
 - ✓ Flash memory has become a powerful and cost-effective solid-state storage technology widely used in mobile electronics devices and other consumer applications.
 - ✓ Two major forms of Flash memory have emerged as the dominant varieties of non-volatile semiconductor memories:
 - NAND Flash
 - NOR Flash

Flash Memory Types

- **NAND Flash** –
 - ✓ Was designed with a very small cell size to enable a **low cost-per-bit of stored data**,
 - ✓ Has been used primarily as a **high-density data storage** for consumer devices such as digital still cameras and USB solid-state disk drives
- **NOR Flash** –
 - ✓ Has typically been **used for code storage and direct execution** in portable electronics devices, such as cellular phones and PDAs.
- Recently, the **distinction between the two types of Flash memory has become less clear.**
 - ✓ Nowadays NAND Flash has become an **attractive alternative for the data storage aspects of today's embedded devices.**
 - ✓ Due to its **higher speed write** and **erase performance** as well as its **low cost-per-bit.**
- As a result, designers of memory subsystems in portable electronics are now using **NAND in some traditional NOR-based applications**

Random Access Memory (RAM) (1)

- Requires external power to maintain memory content.
 - The term random access refers to the ability to access any memory cell directly.
 - RAM is much faster than ROM.
 - Two types of RAM that are of interest:
 - ✓ **Dynamic RAM (DRAM) DRAM** - is a RAM device that requires periodic refreshing to retain its content (new devices nowadays are DDR 3\4 SDRAM).
 - ✓ **Static RAM (SRAM) SRAM** - is a RAM device that retains its content as long as power is supplied by an external power source. SRAM does not require periodic refreshing and it is faster than DRAM.
- **Non-Volatile RAM (NVRAM)** NVRAM is a special type of SRAM that has backup battery power so it can retain its content after the main system power is shut off (i.e, BIOS).

Random Access Memory (RAM) (2)

- **(SDRAM) Synchronous dynamic random access memory** - a DRAM that has a synchronous interface:
 - ✓ Meaning that it **waits for a clock signal before responding** to control inputs and is **therefore synchronized with the computer's system bus**.
 - ✓ The **clock is used to drive an internal finite state machine** that **pipelines incoming instructions**.
 - ✓ This allows the chip to have a more complex pattern of operation than asynchronous DRAM which enables faster I/O.
- **DDR SDRAM (double-data-rate synchronous dynamic random access memory)**
 - ✓ **Achieves nearly twice the bandwidth of the preceding "single data rate" SDRAM.**
 - by **double pumping** (transferring data **on the rising and falling edges** of the clock signal) without increasing the clock frequency.

Random Access Memory (RAM) (3)

- DDR SDRAM gives a transfer rate of $(\text{memory bus clock rate}) \times 2$ (for dual rate) $\times 64$ (number of bits transferred) / 8 (number of bits/byte).
 - ✓ Thus with a bus frequency of 100 MHz, DDR SDRAM gives a maximum transfer rate of 1600 MB/s.
- **DDR2 SDRAM is a double data rate synchronous dynamic random access memory:**
 - Enables the operation of the external data bus at **twice the clock rate**.
 - DDR2 memory **operating at twice the external data bus clock rate** as DDR may provide twice the bandwidth with the same latency (in nanoseconds).

Boot Loader – What is it (1/2)?

- At system power up, the boot image, a program burnt on the targets “ROM” (typically written by hardware engineers) starts execution and does the following:
 1. Initializes the target (the processor and peripherals).
 2. Downloads the Kernel \ user image from the host system to the target.
- The whole process is called **bootstrapping**.
- Most of the early initialization code is in **low-level assembly** language that is **specific to the target systems CPU architecture**.
- Later stage initialization code might be written in a **higher-level programming language, such as C**.

Boot Loader – What is it (2/2)?

- It really has two basic functionalities:
 1. **Boot** - At system power up, the boot image starts execution at the predefined address (**reset vector**), **initializes the target hardware**, as following :
 - ✓ Initializes the CPU into a known state.
 - ✓ Initializes the memory system and the physical RAM,
 - ✓ Initializes the required peripheral devices, for example,
 - The serial interface
 - The system timer
 - The interrupt controller
 - The default interrupt handlers,
 - ✓ It prepares the system to execute the loader.
 2. **Loader** - begins execution **after this boot image** completes, a data transfer utility resides on the host system side.
- The **loader works in conjunction with its host utility** counterpart to perform the image transfer.

Boot loader- Boot stage

- After powered up, the processor **fetches and executes code** from a predefined and hard-wired address offset.
 - ✓ The code contained at this memory location is called the **reset vector**.
 - ✓ The reset vector is usually **a jump instruction into another part** of the memory space where the **real initialization code is found**.
- The **reset vector is contained in a small ROM** (more like **Nor flash nowadays**), which might mapped to location (i.e. 0x0h) of the address space.
- Location of the reset vector is determined during the linking stage while building the image –with respect to hardware configuration.
- Most of the early initialization code is in **low-level assembly** language that is **specific to the target systems CPU architecture**.
- Later stage initialization code might be written in a **higher-level programming language, such as C**.

Boot - Reset Vector example

```
.text
.extern main
.extern CopyProgramFromNand
.extern Uart_Init
__start: .global __start
__start:
B ResetHandler
B HandlerUndef      ;      handler for Undefined mode
B HandlerSWI        ;      handler for SWI interrupt
B HandlerPabort     ;      handler for PAbort
B HandlerDabort     ;      handler for DAbort
B.                  ;      reserved
B HandlerIRQ        ;      handler for IRQ interrupt
B HandlerFIQ        ;      handler for FIQ interrupt
```

ResetHandler:

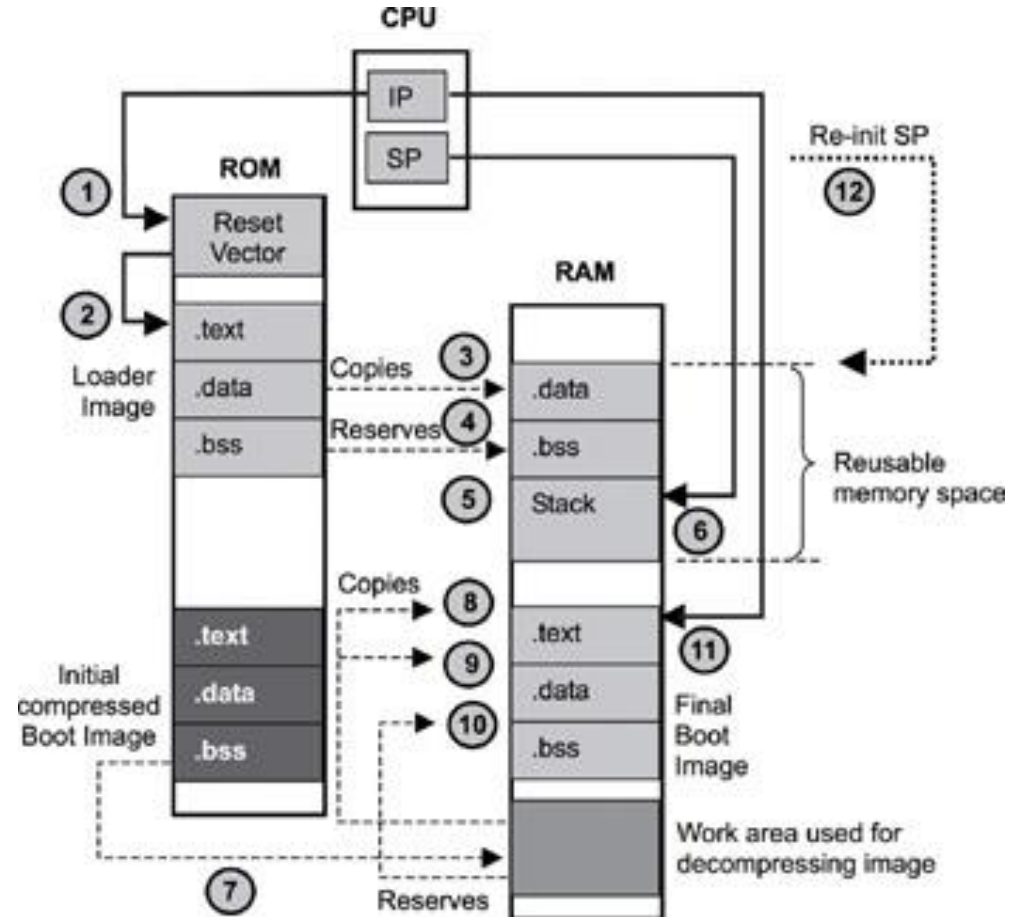
```
ldrr0,=WTCON      /*watch dog disable*/
ldrr1,=0x0
strr1,[r0]
```

```
ldrr0,=INTMSK
ldrr1,=0xffffffff /*all interrupt disable*/
strr1,[r0]
```

```
ldrr0,=INTSUBMSK
ldrr1,=0x7fff/*all sub interrupt disable*/
strr1,[r0]
```

Boot Scenarios – Executing from RAM after Image Transfer from ROM

- CPU executes the **assembly command at the first location** in the Reset Vector (1).
- Image's **data and code is either copied or decompressed to RAM** (3,4,5).
- Stack location is set to a RAM location (6).
- Kernel or user program is **copied to ram** (7)
- **image is decompressed to RAM** (8,9,10)
- **Program counter is set the code fragment in RAM** from which operation proceeds (11).



Boot loader- Loader stage (1/3)

- Once Boot stage is completed – Loader stage is in place.
The loader may: **Load an image that has been already burned into FLASH memory.**
 - ✓ The image might be a **simple user application** or an **O.S. image** (i.e. Embedded Linux). This image is called “**target image**”.
 - ✓ In later case, the boot loader **will automatically unpack the image and then issues a jump to the RAM memory** in which the **unpacked image resides**. This is called “**Auto Restart Mode**”.
 - During the boot loading phase its possible to stop the boot process before loading the O.S.
 - The system will Display a menu that enables the user to issue commands to either diagnose the Hardware or load another external image.

Boot loader- Loader stage (2/3)

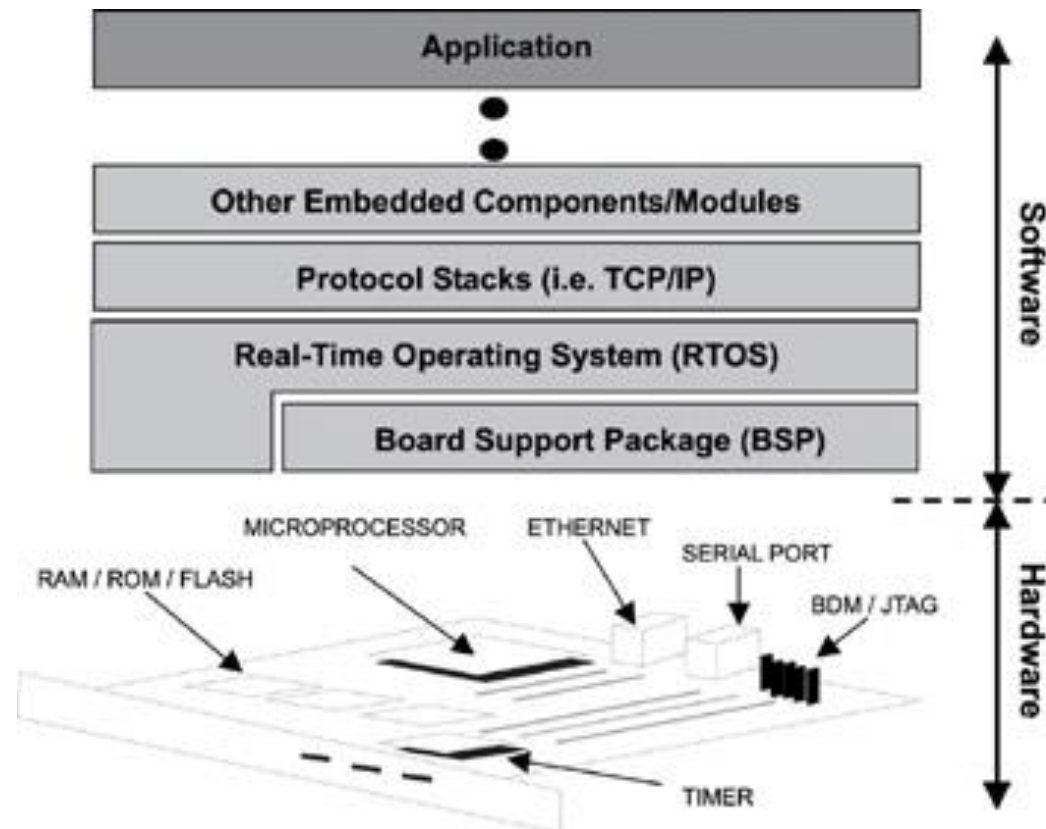
- In order to achieve the downloading a **data transfer protocol**, as well as communication parameters (baud rate for the serial) are agreed upon.
 - ✓ The loader and the utility program operate as a unit, which is often capable of using more than one type of connection.
 - ✓ the transfer might use:
 - Serial Connection (RS232 – very slow)
 - The Network – Ethernet protocol (much faster)
 - A downloading protocol such as the Trivial File Transfer Protocol (TFTP) might be used.
 - USB protocol.

Boot loader- Loader stage (3/3)

- The **target image** is downloaded directly into the RAM memory.
- Sometimes, **the loader needs to understand the object file format** (for example, the ELF format) because, the object file contains information such as the load address, which the loader uses for section placement.
- The **downloaded image might be compressed**, if so **its uncompressed to a different part of memory** and the control is transferred to the downloaded image.
- If the **image is an operating system** – then it **contains the kernel** and the **application code** written by the embedded developer.

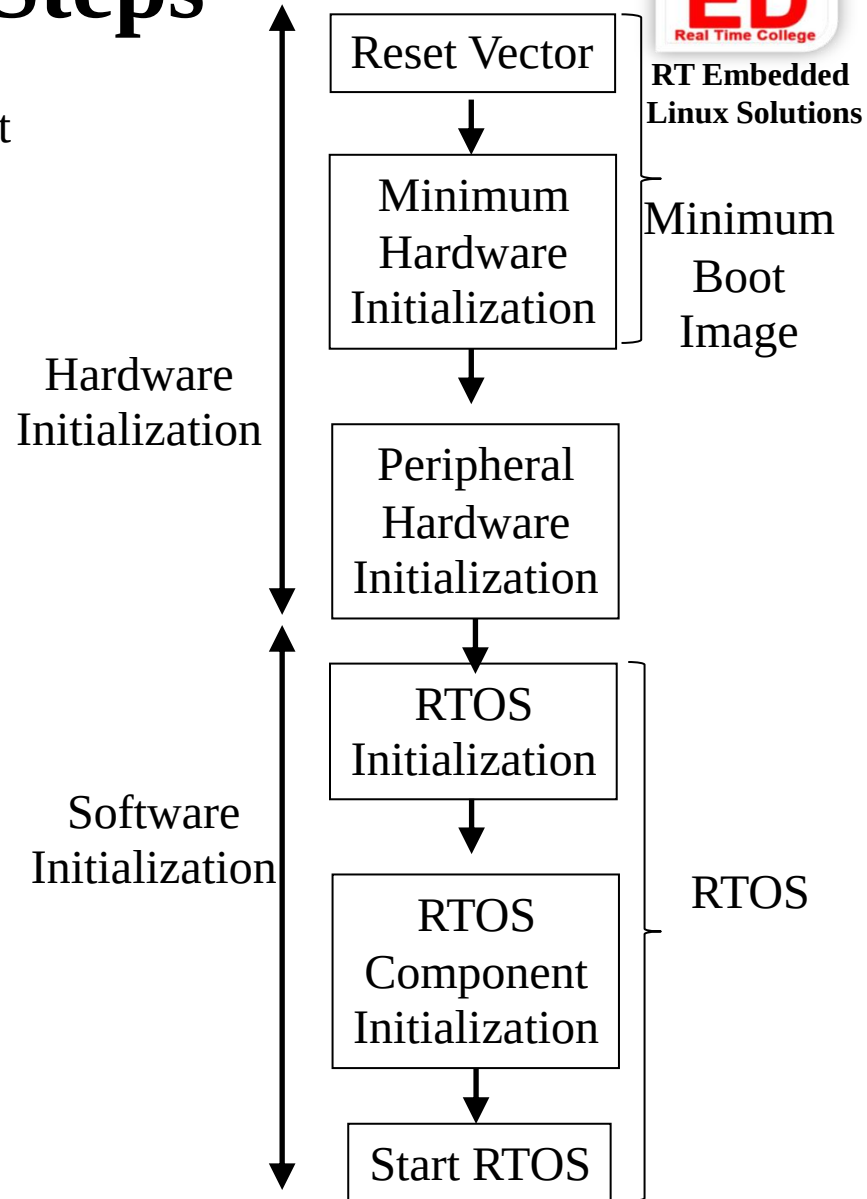
Target Image components

- The target image is a combination of sophisticated software components and modules, including the following:
- **Board Support Package (BSP)** contains a full spectrum of drivers for the system hardware components and devices;
- **RTOS** -provides basic services, such as:
 - ✓ resource synchronization
 - ✓ I/O services,
 - ✓ **scheduling services**
 - ✓ **file system services**
 - ✓ network services.



Target Initialization Steps

- The primary steps required to initialize most target systems are the following:
 - ✓ Hardware Initialization
 - ✓ RTOS Initialization
 - ✓ Application Initialization.



BSP initialization phase

- After the initial execution, the code calls the appropriate functions to initialize other hardware components, if present, on the target system.
- These might include the following:
 - ✓ initializing interrupt handlers;
 - ✓ initializing bus interfaces, such as PCI, and USB;
 - ✓ initializing board peripherals such as serial, LAN, and SCSI
 - ✓ setting up device drivers
- The BSP **provides a set of function calls for upper** software **layers** (for example, the RTOS) to **use in order to communicate with the hardware components** of the target system.

RTOS Initialization

- The RTOS software initialization includes the following:

1. **Initializing the RTOS**

2. Initializing different **RTOS objects and services**, if present :

- ✓ task objects
- ✓ semaphore objects
- ✓ message-queue objects
- ✓ timer services
- ✓ interrupt services
- ✓ memory-management services

3. Creating necessary **stacks for RTOS**

4. initializing additional RTOS extensions, such as:

- TCP/IP stack
File Systems

5. starting the RTOS and its initial tasks (i.e. **Init task**)

Application Software Initialization

- After the **RTOS is initialized and running** with the required components, **control is transferred to a user-defined application**.
 - ✓ The **RTOS code calls a predefined function** which is implemented by the user-defined application (at this point **RTOS services are available**).
 - ✓ This **application also goes through initialization, during which all necessary objects, services, data structures, variables, and other constructs are declared and implemented**.
 - ✓ For a simple, user application such as the hello world application, all the work can be done in this function.
 - ✓ For a **complex application**, it may **create tasks to perform the work**.
 - These application-created tasks will execute once the kernel scheduler runs.
 - The kernel scheduler runs when this control-transfer function exits.

Real-Time Operating Systems

History of Operating Systems



RT Embedded
Linux Solutions

- In the early days of computing, developers created software applications that included **low-level machine code to initialize and interact with the system's hardware directly.**
- This **tight integration between the software and hardware resulted in non-portable applications.**
- A small change in the hardware might result in rewriting much of the application itself.
- Obviously, these systems were difficult and costly to maintain.
- As the software industry progressed, operating systems that provided the basic software foundation for computing systems evolved and facilitated **the abstraction of the underlying hardware from the application code.**
- The evolution of operating systems helped **shift the design of software applications from large, monolithic applications** to more **modular, interconnected applications that could run on top of the operating system environment.**

History of Operating Systems

- Over the years, many versions of operating systems evolved.
- These ranged from
 - general-purpose operating systems (GPOS), such as UNIX and Microsoft Windows,
 - to smaller and more compact real-time operating systems, such as VxWorks.
- In the 60s and 70s, when mid-sized and mainframe computing was in its prime, UNIX was developed to facilitate multi-user access to expensive, limited-availability computing systems. UNIX allowed many users performing a variety of tasks to share these large and costly computers. multi-user access was very efficient:
- In the 80s, Microsoft introduced the Windows operating system, which emphasized the personal computing environment. Targeted for residential and business users interacting with PCs through a graphical user interface, the Microsoft Windows operating system helped drive the personal-computing era.

Modern Operating Systems – General characteristics.



RT Embedded
Linux Solutions

- Later in the decade, momentum started building for the next generation of computing: the post-PC, **embedded-computing era**.
- To meet the needs of **embedded computing**, **commercial RTOSs**, such as **VxWorks**, were developed.
- Nowadays the Embedded Linux O.S. has taken over
- RTOSs and GPOSs have some functional similarities such as:
 - some level of **multitasking**,
 - software and hardware **resource management**,
 - provision of **underlying OS services** to applications
 - **abstracting the hardware** from the software application.

Real time Operating Systems – Key characteristics.



RT Embedded
Linux Solutions

Key [functional differences](#) that set RTOSs apart from GPOSs include:

- **Predictability** - completion of operating system calls occurs within known timeframes.
- **Reliability** - the system might need to operate for long periods without human intervention.
- **Scalability** - depending on how much functionality is required, an RTOS should be capable of adding or deleting modular components, including file systems and protocol stacks.
- **Performance** - must perform fast enough to fulfill its timing requirements.
- **Compactness** – fit in with size requirements imposed by memory limitations, which are dictated by demand for low cost.

What's POSIX?

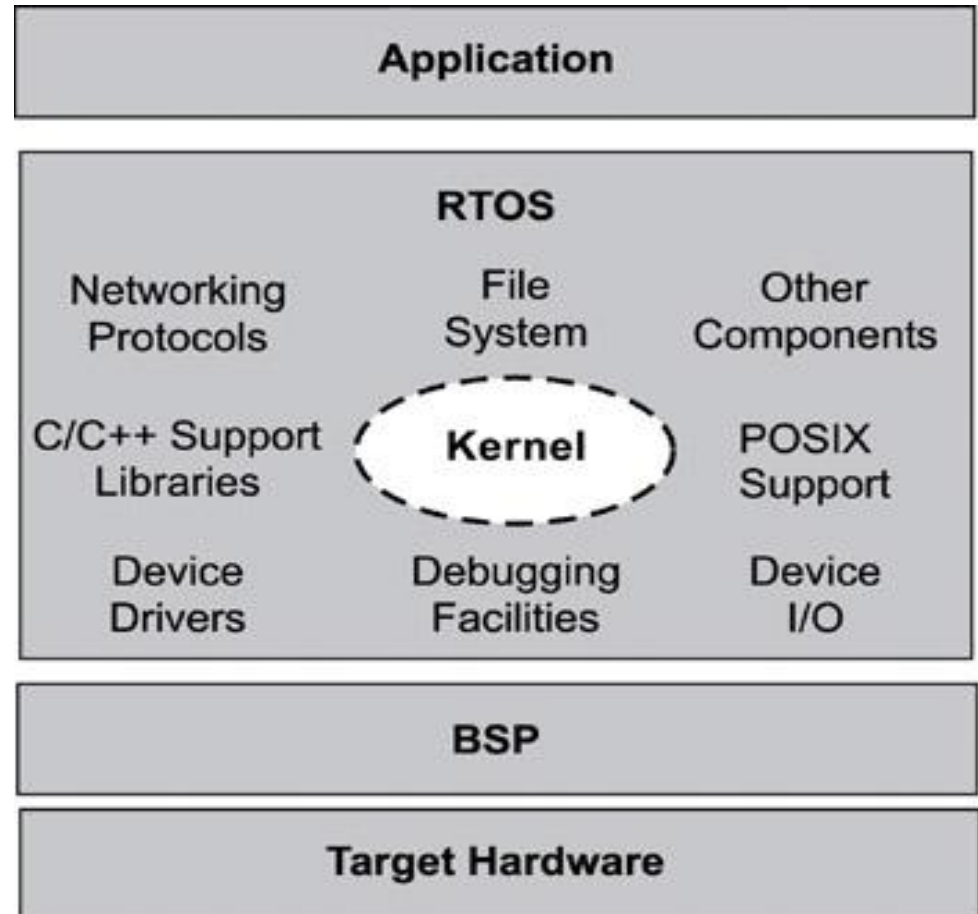
- POSIX, the **Portable Operating System Interface**, is an evolving, growing document that is being produced by the IEEE and standardized by ANSI and ISO.
- The goal of POSIX is the **source-code portability** of applications:
- it makes **porting** (moving an application from one operating system or machine to another) **by simply recompiling** it.
- **This goal is partly unattainable** since most applications, especially the real-time ones, **require more operating system support than you can find** in any particular standard.
- However, POSIX has achieved a less apocalyptic version of this goal:
- reducing the amount of effort needed for porting.

RTOS Components

A real-time operating system (RTOS) is a program that

- **schedules execution in a timely manner,**
- **manages system resources,**
- **provides a consistent foundation for developing application code.**

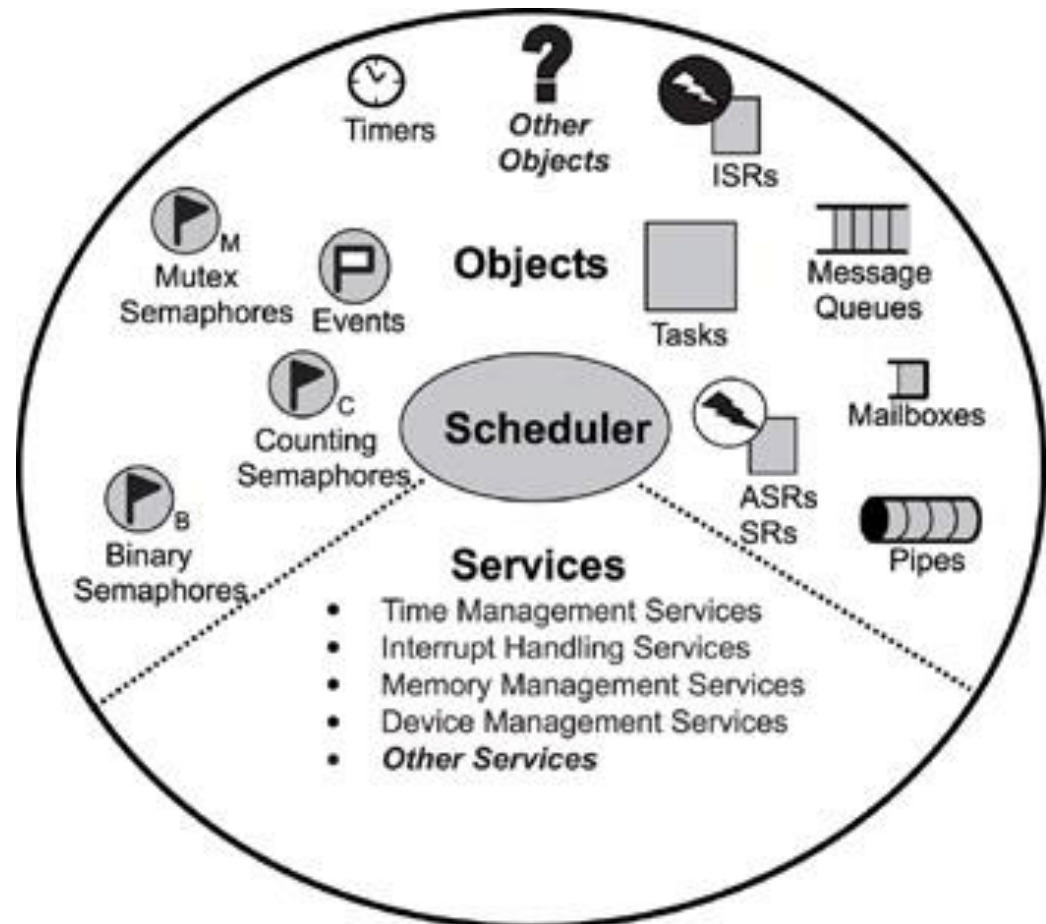
Application code designed on an RTOS can be quite diverse, therefore the RTOS should be scalable in order to meet different sets of requirements for different applications.



RTOS Components

Common components in
an RTOS kernel:

- **the scheduler,**
- **objects**
- **services**



RTOS Components

- **Scheduler** - is contained within each kernel and follows a set of algorithms that determines which task executes when. Some common examples of scheduling algorithms include **round-robin** and **preemptive scheduling**.
- **Objects** - are special kernel constructs that help developers create applications for real-time embedded systems. Common kernel objects include **tasks**, **semaphores**, and **message queues**.
- **Services** - are operations that the kernel performs on an object or, generally operations such as timing, interrupt handling, and resource management. For example:
 - The service “**taskCreate**” can be issued on the object “**task**”.
 - The service “**semTake**” can be issued on the object “**semaphore**”.

The Scheduler

- The scheduler is at the heart of every kernel.
- A scheduler provides the algorithms needed to determine which task executes when.
- In order to understand how scheduling works, the following topics will be describes:
 - schedulable entities,
 - multitasking,
 - context switching,
 - dispatcher,
 - scheduling algorithms.

Scheduler: Schedulable Entities

- A schedulable entity is a kernel object that can **compete for execution time** on a system, **based on a predefined scheduling algorithm** (such as Tasks, processes, Threads)
- **Threads** - The smallest units of processing that can be scheduled by an operating system.
- **Tasks** - Entities that provide management “envelope” to one thread each.
- **Processes** - Entities that provide management “envelope” to at least one thread each, but generally manage many threads that share the same “envelope” resources.
- In contrary to tasks – these are isolated entities. Threads in one process can’t directly access resources that belong to threads in another process. This memory protection feature comes at the expense of performance overhead.
- There are versions of Real-Time operating systems such as VxWorks that provide the Software Designer with the option to **use tasks entities instead of processes**. In that case – **there is no isolation among them – which makes the system risky**.
- How exactly does a scheduler handle multiple schedulable entities that need to run simultaneously? The answer is by multitasking

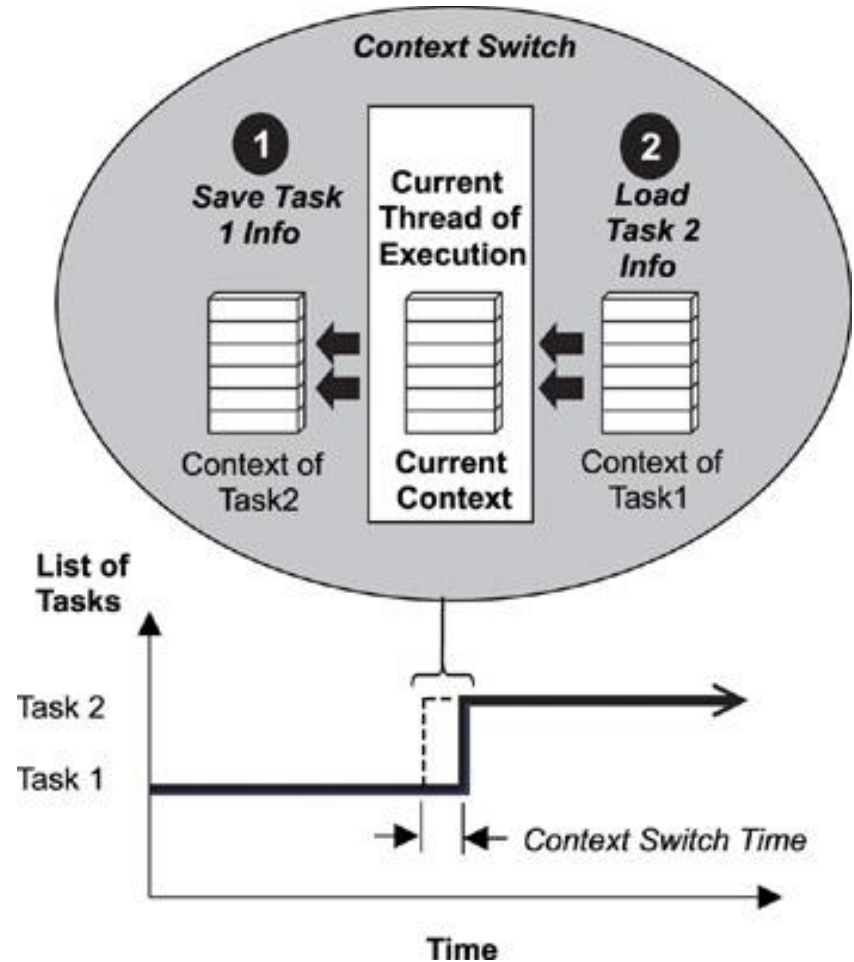
Scheduler: Multitasking

Multitasking is the ability of the operating system to **handle multiple activities within set deadlines**.

A real-time kernel might have multiple tasks that it has scheduled to run

The kernel multitasks in such a way that **many threads of execution appear to be running concurrently**; however, the kernel is actually **interleaving executions sequentially**, based on a preset **scheduling algorithm**.

The scheduler must ensure that the appropriate task runs at the right time. **Context switching** itself is **time consuming** and requires resources.



Scheduler: Context Switching

- Each **task has its own context**, which is **the state of the CPU registers** required each time it is scheduled to run.
- A **context switch** **occurs when the scheduler switches from one task to another**.
- Every time a new **task is created**, the **kernel also creates and maintains an associated task control block (TCB)**.
- TCB is a system data structure that maintain task-specific information (Such as?).
- when the kernels **scheduler determines that it needs to stop running task 1 and start running task 2**, it takes the following steps:
 1. The kernel saves task 1's context information in its TCB.
 2. It loads tasks 2 context information from its TCB, which becomes the current thread of execution.
 3. The context of task 1 is frozen while task 2 executes, but if the scheduler needs to run task 1 again, task 1 continues from where it left off just before the context switch.

Scheduler: Context Switching

- The time it takes for the scheduler to switch from one task to another is the context switch time.
- It is relatively insignificant compared to most operations that a task performs.
- If an applications design includes frequent context switching, however, the application can incur unnecessary performance overhead.
- Every time an application makes a system call, the scheduler has an opportunity to determine if it needs to switch contexts.
- When the scheduler determines a context switch is necessary, it relies on an associated module, called the dispatcher, to make that switch happen.

Scheduler: The Dispatcher

- The dispatcher is the part of the scheduler that performs the Actual context switching and changes the flow of execution.
- At any time an RTOS is running, the flow of execution, can be in of three states:
 - Application Task (User thread)
 - A Kernel System call
 - An ISR
- There is no difference in priority between user and kernel threads.
- When a task or ISR makes a system call, the flow of control passes to the kernel to execute one of the system routines provided by the kernel.
- When it is time to leave the kernel, the dispatcher is responsible for passing control to one of the tasks in the user's application.
 - It will not necessarily be the same task that made the system call.
 - It is the scheduling algorithms of the scheduler that determines which task executes next.
 - It is the dispatcher that does the actual work of context switching and passing execution control.

Tasks and the Scheduler:

Task Scheduling Algorithms

The scheduler determines which task runs by following a [scheduling algorithm](#) (also known as scheduling policy).

Most kernels today support two common scheduling algorithms:

- [Preemptive Priority-Based Scheduling](#).
- [Round-Robin Scheduling](#).

The RTOS manufacturer typically predefines these algorithms; however, in some cases, [developers can create and define their own scheduling](#) algorithms (not so easy!!!!).



**RT Embedded
Linux Solutions**

Tasks

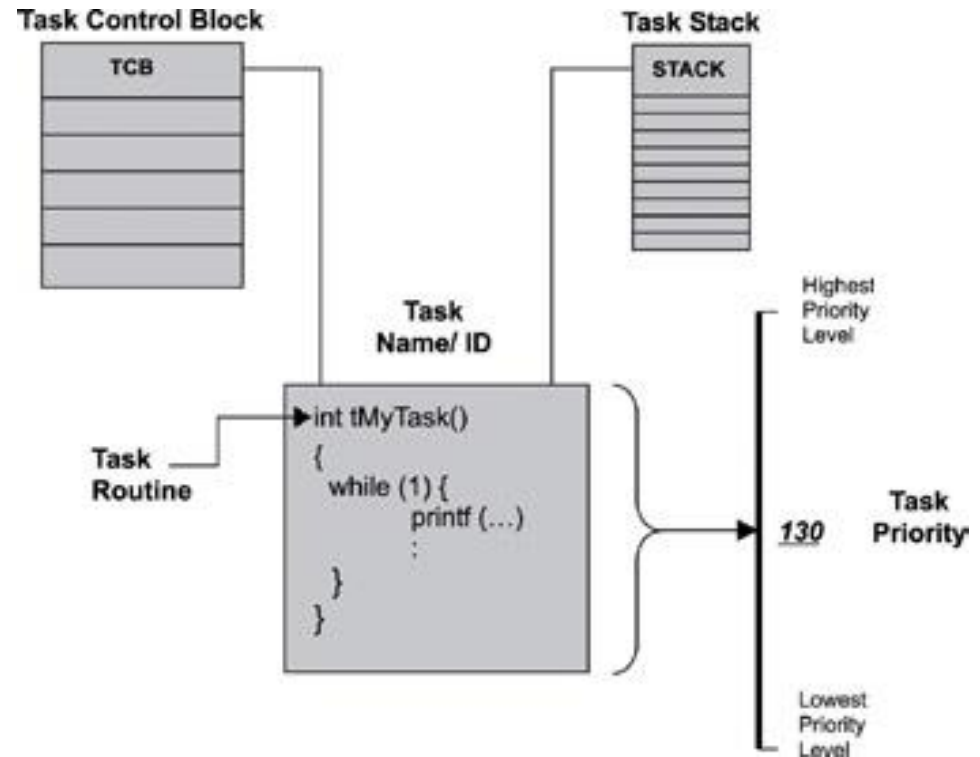
Defining a Task

- A task is **an independent thread of execution** that can **compete with other concurrent tasks** for processor execution time.
- Developers decompose **applications into multiple concurrent tasks** to **optimize** the handling of **inputs and outputs** within set time constraints.
- A task is able to compete for execution time on a system, based on a predefined scheduling algorithm.
- A task is defined by its distinct set of parameters and supporting data structures

Defining a Task

Specifically, upon creation, each task has:

- an associated name
- unique ID
- priority
- task control block (TCB)
 - Task state
 - CPU usage statistics
 - List of handles.
 - Base Priority.
 - Signal Handlers.
- stack,
- task routine.



Tasks and the Scheduler:

Preemptive Priority-Based Scheduling

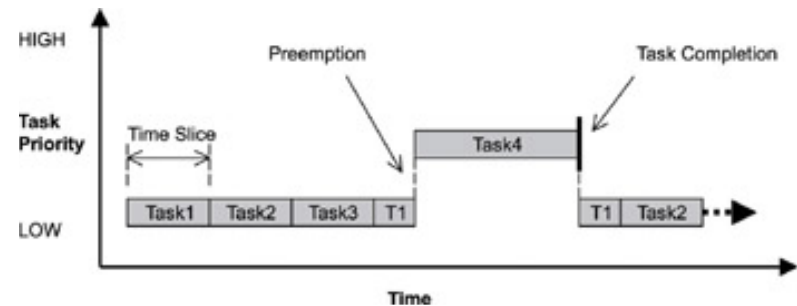
- Most Real-Time kernels use preemptive priority-based scheduling by default.
 - The task that gets to run at any point is the task with the highest priority among all other tasks ready to run in the system.
- Real-time kernels generally support 256 priority levels, in which 0 is the highest and 255 the lowest (Linux is different).
- With a preemptive priority-based scheduler, each task has a priority, and the highest-priority task runs first (the priority can change to support Real Time).
- If a task with a priority higher than the current task becomes ready to run, the kernel immediately saves the current task's context in its TCB and switches to the higher-priority task

Tasks and the Scheduler:

Round-Robin Scheduling

- Round-robin scheduling provides each task an **equal share of the CPU execution time**.

- Pure round-robin scheduling cannot satisfy real-time system requirements because in real-time systems, tasks perform work of varying degrees of importance.



- With time slicing, **each task executes for a defined interval**, or time slice, in an ongoing cycle.
- Newly added tasks are placed **at the end of the cycle**, with their run-time counters initialized to 0.

* Round-robin **Tasks per priority** ????????

System Tasks

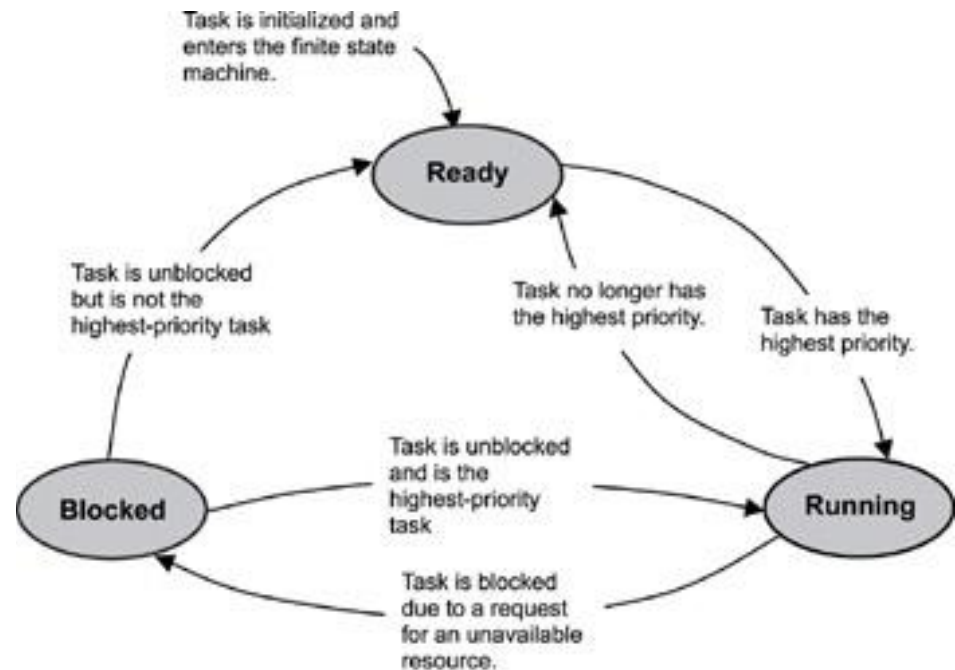
- When the kernel first starts, it **creates its own set of system tasks** and allocates the **appropriate priority** for each from a set of **reserved priority levels**.
- The reserved priority levels refer to the priorities **used internally by the RTOS** for its system tasks.
- An **application should avoid using these priority levels** for its tasks because running application tasks at such level may affect the overall system performance or behavior.
- Examples of system tasks include:
 - initialization or startup task** - initializes the system and creates and starts system tasks.
 - idle task** - uses up processor idle cycles when no other activity is present.
 - logging task** - logs system messages,
 - exception-handling task** - handles exceptions,
 - debug agent task** - allows debugging with a host debugger.

Idle task

- The idle task is **set to the lowest priority** (Why), typically executes in an **endless loop**, and **runs when either no other task can run** or when **no other tasks exist**, for the sole purpose of using idle processor cycles.
- The idle task is necessary because the processor executes the instruction to which the program counter register points while it is running.
- Unless the processor can be suspended, the **program counter must still point to valid instructions** even when no tasks exist in the system or when no tasks can run.
- The kernel might allow a **user-configured routine to run instead of the idle task**. One example of a special requirement is **power conservation**.
 - When no other tasks can run, **the kernel can switch control to the user-supplied routine** instead of to the idle task.
- the user-supplied routine acts like the idle task but instead **initiates power conservation code, such as system suspension**, after a period of idle time.

Task States and Scheduling

- At any time each **task exists in one of a small number of states**:
 - Ready**,
 - Running**,
 - Blocked**.
- As the real-time embedded system runs, each task moves from one state to another.



Ready State - the task is ready to run but **cannot because a higher priority task is executing**.

Blocked State - the task has **requested a resource that is not available**, it will wait until some event occurs, or has delayed itself for some duration.

Running State - the **task is the highest priority task and is running**..

Task States and Scheduling

Some commercial kernels, such as the VxWorks kernel, define other, more granular states, such as:

- *suspended* - exists for debugging purposes.
- *pending* - task is waiting for a resource that it needs to be freed.
- *delayed* - task is waiting for a timing delay to end.

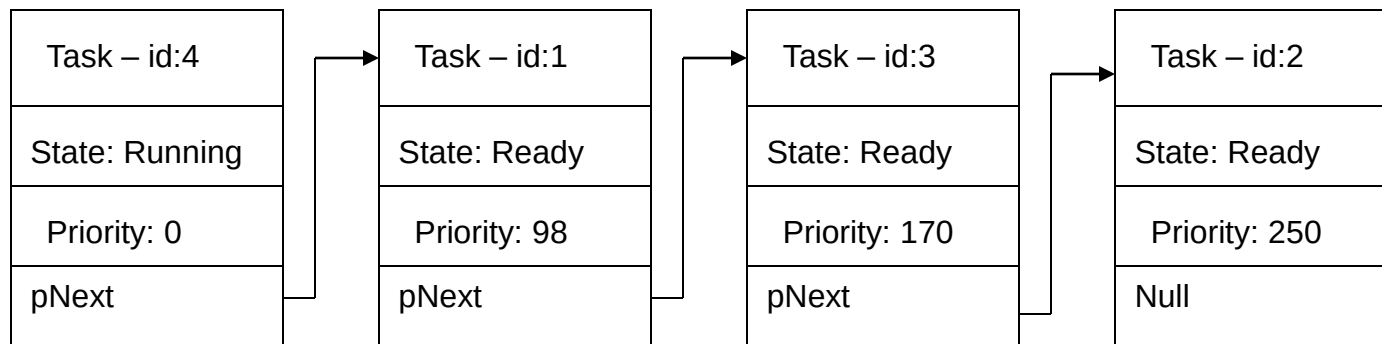
Ready State

- When a task is first created and made ready to run, the kernel **puts it into the ready state**.
 - In this state, the **task actively competes with all other** ready tasks for the processor's execution time.
- The tasks in the **ready state cannot move directly to the blocked state**.
 - A **task first needs to run so it can make a blocking call**, which is a call to a function that cannot immediately run to completion, thus putting the task in the blocked state.
- Ready tasks, therefore, **can only move to the running state**.
- The kernel's scheduler uses the **priority of each task to determine which task to move to the running state**.
 - most kernels **support more than one task per priority level**, allowing many more tasks in an application.
 - In this case, the scheduling algorithm is more complicated and involves maintaining a task-ready list . Some kernels maintain a separate task-ready list for each priority level; others have one combined list.

Ready State

The scheduler uses a **data structure called the ready list** to track the tasks that are in the ready state.

the ready list **is implemented as an ordinary linked list**, ordered by priority. So the **head of this list is always the highest priority task** that is ready to run.



Kernel Scheduler Example

five-step scenario of how a kernel scheduler might use a task-ready list to move tasks from the ready state to the running state.

This example assumes a **single-processor system and a priority-based preemptive scheduling algorithm in which 255 is the lowest priority and 0 is the highest**

1. Tasks 1, 2, 3, 4, and 5 are ready to run and are waiting in the task-ready list.

1 First-Step: State of Task-Ready List

Task 1 Priority=70	Task 2 Priority=80	Task 3 Priority=80	Task 4 Priority=80	Task 5 Priority=90
-----------------------	-----------------------	-----------------------	-----------------------	-----------------------

2 Second-Step: State of Task-Ready List

Task 2 Priority=80	Task 3 Priority=80	Task 4 Priority=80	Task 5 Priority=90	
-----------------------	-----------------------	-----------------------	-----------------------	--

3 Third-Step: State of Task-Ready List

Task 3 Priority=80	Task 4 Priority=80	Task 5 Priority=90		
-----------------------	-----------------------	-----------------------	--	--

4 Fourth-Step: State of Task-Ready List

Task 4 Priority=80	Task 5 Priority=90			
-----------------------	-----------------------	--	--	--

5 Fifth-Step: State of Task-Ready List

Task 4 Priority=80	Task 2 Priority=80	Task 5 Priority=90		
-----------------------	-----------------------	-----------------------	--	--

Kernel Scheduler Example

2. task 1 has the highest priority (70), kernel removes task 1 from the ready list and moves it to the running state.

3. During execution, task 1 makes a blocking call, the kernel moves task 1 to the blocked state; task 2, moves to the running state.

4. task 2 makes a blocking call. The kernel moves task 2 to the blocked state; moves task 3, to the running state.

1 First-Step: State of Task-Ready List

Task 1 Priority=70	Task 2 Priority=80	Task 3 Priority=80	Task 4 Priority=80	Task 5 Priority=90
-----------------------	-----------------------	-----------------------	-----------------------	-----------------------

2 Second-Step: State of Task-Ready List

Task 2 Priority=80	Task 3 Priority=80	Task 4 Priority=80	Task 5 Priority=90	
-----------------------	-----------------------	-----------------------	-----------------------	--

3 Third-Step: State of Task-Ready List

Task 3 Priority=80	Task 4 Priority=80	Task 5 Priority=90		
-----------------------	-----------------------	-----------------------	--	--

4 Fourth-Step: State of Task - Ready List

Task 4 Priority=80	Task 5 Priority=90			
-----------------------	-----------------------	--	--	--

5 Fifth-Step: State of Task-Ready List

Task 4 Priority=80	Task 2 Priority=80	Task 5 Priority=90		
-----------------------	-----------------------	-----------------------	--	--

Kernel Scheduler Example

5. As task 3 runs, frees the resource that task 2 requested. The kernel returns task 2 to the ready state and inserts it at the end of the list of tasks ready to run at priority level 80. Task 3 continues as the currently running task..

1 First-Step: State of Task-Ready List

Task 1 Priority=70	Task 2 Priority=80	Task 3 Priority=80	Task 4 Priority=80	Task 5 Priority=90
-----------------------	-----------------------	-----------------------	-----------------------	-----------------------

2 Second-Step: State of Task-Ready List

Task 2 Priority=80	Task 3 Priority=80	Task 4 Priority=80	Task 5 Priority=90	
-----------------------	-----------------------	-----------------------	-----------------------	--

3 Third-Step: State of Task-Ready List

Task 3 Priority=80	Task 4 Priority=80	Task 5 Priority=90		
-----------------------	-----------------------	-----------------------	--	--

4 Fourth-Step: State of Task - Ready List

Task 4 Priority=80	Task 5 Priority=90			
-----------------------	-----------------------	--	--	--

5 Fifth-Step: State of Task-Ready List

Task 4 Priority=80	Task 2 Priority=80	Task 5 Priority=90		
-----------------------	-----------------------	-----------------------	--	--

Running State

On a **single-processor system**, **only one task can run at a time**.

- when a **task is moved to the running state**, the **processor loads its registers with this task's context**.
 - The processor can then execute the task's instructions and manipulate the associated stack.
- A task can **move back to the ready state** while it is running.
 - In this case **it is preempted by a higher priority task**. the preempted task is put in the appropriate, priority-based location in the task-ready list, and the higher priority task is moved from the ready state to the running state.
- A **running task can move to the blocked state** in any of the following ways:
 - by making a call that **requests an unavailable resource**,
 - by making a call that **requests to wait for an event to occur**, and
 - by making a call to **delay the task for some duration**.

Blocked State

The possibility of **blocked states** is extremely important in real-time systems because **without blocked states, lower priority tasks could not run.**

- If **higher priority tasks are not designed to block**, **CPU starvation can result**
 - CPU starvation occurs when higher priority tasks use all of the CPU execution time and lower priority tasks do not get to run.
- A task can only move to the blocked state by making a blocking call, requesting that some blocking condition be met.
- A blocked task remains blocked until the blocking condition is met. (It probably ought to be called the unblocking condition).

Blocked State

- Examples of how **blocking conditions are met (task is unblocked)** include the following:
 - a **semaphore (a lock for now)** for which a task is waiting is released.
 - a **message, on which the task is waiting, arrives** in a message queue.
 - a **time delay imposed** on the task expires.
- When a **task becomes unblocked**, the task might **move from the blocked state to the ready state** if **it is not the highest priority task** .
 - The task is then put into the task-ready list at the appropriate priority-based location.
- if the **unblocked task is the highest priority** task, the **task moves directly to the running state** (without going through the ready state- NOT in Linux) and preempts the currently running task.
 - The preempted task is then moved to the ready state and put into the appropriate priority-based location in the task-ready list.

Typical Task Operations

- In addition to providing a task object, kernels also provide **task-management services** .
 - Task-management services include the actions that a kernel performs behind the scenes to support tasks, for example, **creating and maintaining the TCB and task stacks**.
- Kernels also provide APIs that allows developers to manipulate tasks.
 - Some operations that developers can perform with a task object from within the application include:
 - ✓ **creating and deleting tasks,**
 - ✓ **controlling task scheduling**
 - ✓ **obtaining task information.**

Task Deletion

- Carefully consider how tasks are deleted (specially in embedded application)
During the deletion process, a kernel terminates the task and frees memory by deleting the task's TCB and stack.
- When tasks execute, they can acquire memory or access resources using other kernel objects.
 - If the task is deleted incorrectly, the task might not get to release these resources.
- For example, assume that a task acquires a semaphore token to get exclusive access to a shared data structure. when the task gets deleted. If not handled appropriately, this abrupt deletion of the operating task can result in:
 - a corrupt data structure, due to an incomplete write operation,
 - an unreleased semaphore, which will not be available for other tasks that might need to acquire it,
 - an inaccessible data structure, due to the unreleased semaphore.
- As a result, premature deletion of a task can result in memory or resource leaks.

Memory \ Resource Leak

Memory Leak - occurs when **memory is acquired but not released**, which **causes the system to run out of memory** eventually.

Resource Leak - occurs when a **resource is acquired but never released**, **which results in a memory leak** because each resource takes up space in memory.

Setting Special Task States

From the time a task is created to the time it is deleted, the task can move through various states (much of them changing automatically).

kernels provide a set of API calls that allow developers to control when a task moves to a different state:

- **Suspend** - Suspends a task
- **Resume** - Resumes a task
- **Delay** - Delays a task
- **Restart** - Restarts a task
- **Set Priority** - Dynamically sets a task's priority
- **Preemption lock** - Locks out higher priority tasks from preempting the current task
- **Preemption unlock** - Unlocks a preemption lock

Tasks in FreeRTOS

Managing Tasks

- A task is a small program in its own right competing for CPU time.
 - its own copy of local variables (not true if variable is declared static in which case only one copy shared by all would exist)
- FreeRTOS tasks will normally run forever within an infinite loop.
 - ☐ Not allowed to return from implementing function.
 - ☐ No 'return' statement.
- If a task is no longer required, it should be deleted.
 - ☐ Should the task implementation ever break out of the above loop, then the task must be deleted before reaching the end its implementing function.
 - ☐ The NULL parameter passed to the `vTaskDelete()` API function indicates the task to be deleted is the calling (this) task (convention used)

Creating Tasks - *xTaskCreate()* API (1)

- Tasks are created using the FreeRTOS *xTaskCreate()* API function.

```
BaseType_t xTaskCreate( TaskFunction_t pvTaskCode,  
                        const char * const pcName,  
                        uint16_t usStackDepth,  
                        void *pvParameters,  
                        UBaseType_t uxPriority,  
                        TaskHandle_t *pxCreatedTask );
```

- There are two possible return values:
 1. *pdPASS* - indicates that the task has been created successfully.
 2. *pdFAIL* - indicates that the task has not been created.

Due to - insufficient heap memory.

Creating Tasks - *xTaskCreate()* API (2)

Parameter name	Description
<i>pvTaskCode</i>	pointer to the function that implements the task
<i>pcName</i>	A descriptive name for the task. Not used by FreeRTOS kernel, purely as a debugging aid. Identifying a task by a human readable name.
<i>usStackDepth</i>	tells the kernel how large task's stack should be. The value specifies the number of words Not bytes.
<i>pvParameters</i>	the value passed into the task
<i>uxPriority</i>	Defines the priority at which the task will execute. Priorities can be assigned from 0, which is the lowest priority, to (configMAX_PRIORITIES – 1), which is the highest priority.
<i>pxCreatedTask</i>	handle to the task being created. If your application has no use for it, set to NULL

Task Example

```
void TaskFunction( void *pvParameters )  
{  
    int32_t lVariableExample = 0;  
  
    /* A task will normally be implemented as an infinite loop. */  
    for( ;; )  
    {  
        /* The code to implement the task functionality  
        will go here. */  
    }  
  
    vTaskDelete( NULL );  
}
```

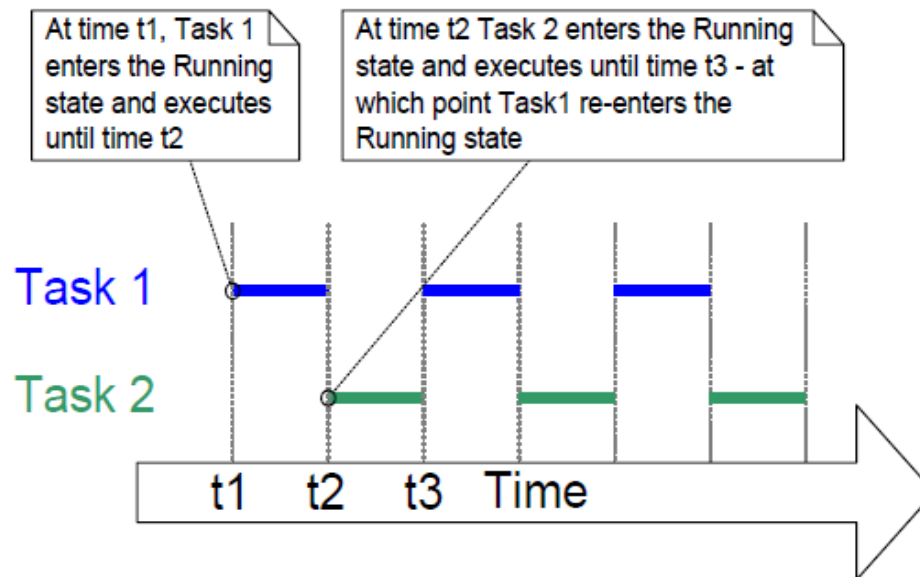
Practical lab

Creating tasks

Task Scheduling

Assume we have created 2 tasks running concurrently on the same CPU
What would their scheduling look like ?

- They would both be entering and exiting the Running state (CPU).
- Both tasks would be running at the same priority, so they would share equal time-slices (round robin scheduling) on the same processor core.



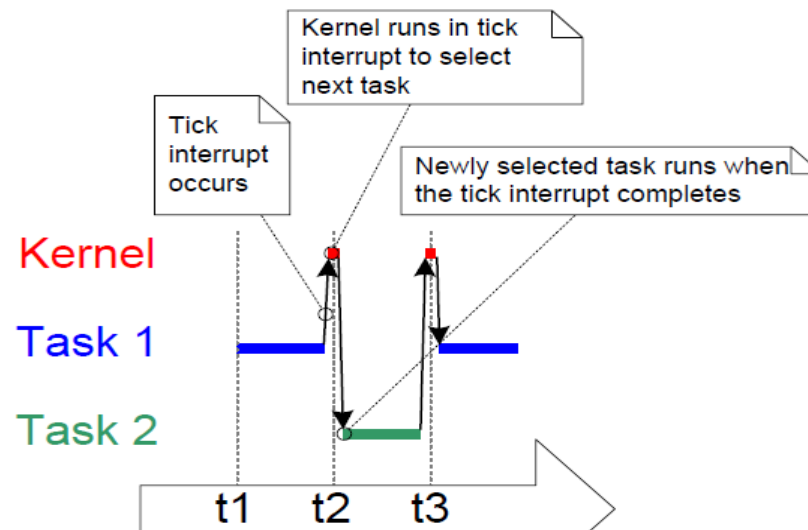
The actual execution pattern of the two Example 1 tasks

Task Priorities

- The **uxPriority** parameter of the *xTaskCreate()* API function assigns an initial priority to the task being created.
- The **priority can be changed** after the scheduler has been started by using the *vTaskPrioritySet()* API function.
- The maximum number of priorities available is set by the application-defined **configMAX_PRIORITIES** compile time configuration within FreeRTOSConfig.h.
- priority is the lowest priority possible, range of available priorities is from 0 to (configMAX_PRIORITIES – 1).
- Any number of tasks can share the same priority—ensuring maximum design flexibility.
- The scheduler will always ensure that **the highest priority task that is able to run is the task selected** to enter the Running state.

the Tick Interrupt

- In order to be able to select the next task to run, the scheduler must run as well (at the end of each time slice).
- A periodic interrupt, called the ‘tick interrupt’, is used for this purpose.
- The **length of the time slice is set** by the **tick interrupt frequency**, which is configured by the application-defined `configTICK_RATE_HZ` **compile time configuration constant** within `FreeRTOSConfig.h`.
- For example, if `configTICK_RATE_HZ` is set to 100 (Hz), then the time slice will be 10 milliseconds
- The time between two tick interrupts is called the ‘tick period’.



pdMS_TO_TICKS() macro

- FreeRTOS API calls specify time in multiples of tick periods.
- The `pdMS_TO_TICKS()` macro converts a time specified in milliseconds into a time specified in ticks.
- The resolution available depends on the defined tick frequency, and `pdMS_TO_TICKS()` cannot be used if the tick frequency is above 1KHz (if `configTICK_RATE_HZ` is greater than 1000).

Note:

- It is not recommended to specify times in ticks directly within the application, but instead to use the `pdMS_TO_TICKS()` macro to specify times in milliseconds, and in so doing, ensure times specified does not change (based on tick frequency).

Designing Threaded Programs

Threads- What are they ?

Threads are the smallest unit of processing that can be scheduled by an operating system.

Threads provide an easier and more efficient means of dealing with multiple concurrent operations.

Every Operating System uses Threads.

Design considerations

When and where should we use threads (in our program)?

Whenever a task has one or more of these properties, consider running it in a thread:

1. potential parallelism
2. overlapping I/O
3. asynchronous events,
4. real-time scheduling

Design considerations – Cont.

use the following criteria:

is it independent of other tasks:

- Does the task **use separate resources than other tasks?**
- Does **its execution depend on the results of other tasks?**
- Do **other tasks depend on its results?**
 - The more tasks depend on each other and share resources, the more the threads executing them will end up blocked waiting on each other

can it become blocked in potentially long waits.

- **Can the task spend a long time in a blocked state?**
- A program can **typically perform millions of integer operations in the time it would take to perform a single I/O operation.**
 - If you dedicate a thread to the I/O task, the rest of the program could accomplish a lot more work in less time.

Design considerations – Cont.

should It use a lot of CPU cycles.

- Does the task **perform long computations**, such as matrix crunching, hashing, or encryption?
 - Time-consuming calculations that are independent of activities elsewhere in the program **are good candidates for threading**.
 - you might let **a thread executing on one CPU process a long computation while other threads on other CPUs handle input**.

must it respond to asynchronous events.

- Must the **task handle events that occur at random intervals**, such as network communications or interrupts from hardware and the operating system?
 - Use **threads to encapsulate and synchronize the servicing of these events, apart** from the rest of your application.

Models

Over time a few **common models for threaded programs** have emerged. These **models define how a threaded application delegates its work** to its threads and how the threads intercommunicate.

None of the models have been explicitly designed for a specific type of application, but each **model tends to be better suited** than the others for certain types.

The following Models discussed are :

- The **boss/worker** model
- The **peer** model
- The **pipeline** model

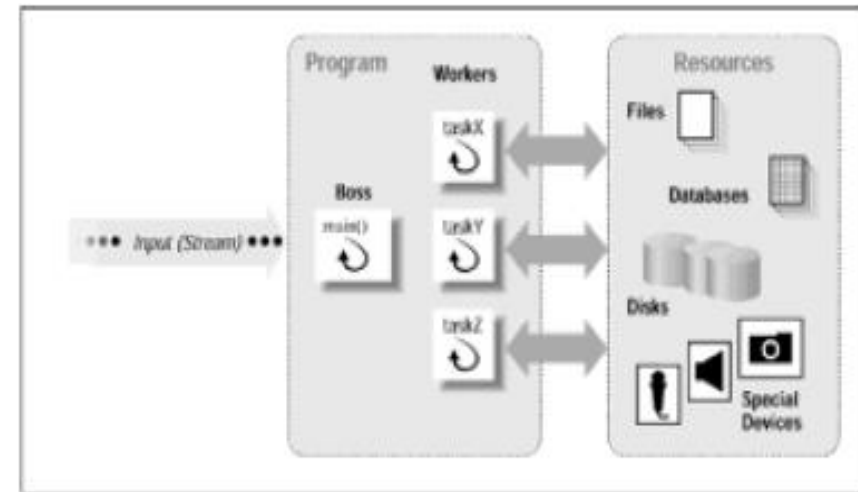
Boss/Worker Model

A Boss thread, **accepts input for the entire program**. it **passes off specific tasks to one or more worker threads**.

The boss **creates each worker thread**, **assigns it tasks**, and, if necessary, **waits for it to finish**.

After creating each worker, the boss returns to the top of its loop to process the next request. **If no requests are waiting, the boss loops until one arrives.**

Once finished, each worker thread can be made responsible for any output resulting from its task, or let the Boss handle its output.



Boss/Worker – Implementation 2 Example

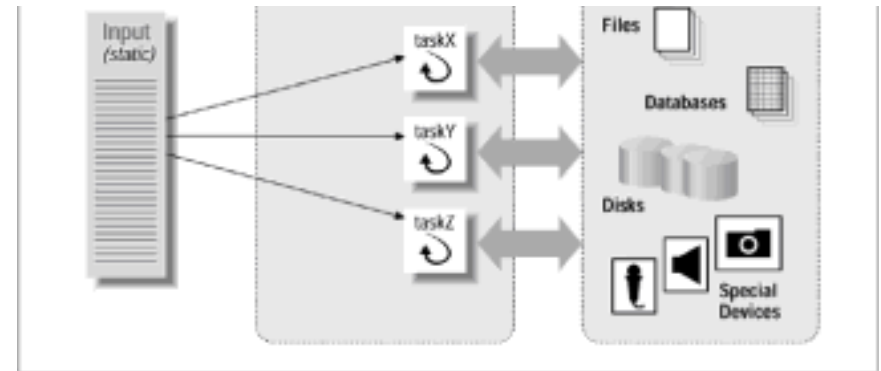
```
main()
/* The boss */
{
    for (i=0; i<the number of workers; )
        thread_create( ... pool_base )
    forever
    {
        get a request
        extract the queue ID to which this request is dedicated.
        place request in the dedicated thread's work queue
        signal the dedicated thread that work is available
    }
}

dedicated_thread_i()
{
    forever
    {
        sleep until data is available in the dedicated queue
        fetch a work request from the dedicated queue.
        process_dedicated_request ()
    }
}
```

Peer Model

all threads work concurrently on their tasks without a specific leader.

one thread must create all the other peer threads when the program starts. However, unlike the boss/worker model, this thread subsequently acts as just another peer thread that processes requests, or suspends itself waiting for the other peers to finish.



The peer model makes each thread responsible for its own input. A peer knows its own input ahead of time, has its own private way of obtaining its input, or shares a single point of input with other peers.

Peer Model – Shard Mem Example

```

Creator_thread()
{
    thread_create( ... thread1 ... task1 )
    thread_create( ... thread2 ... task2 )
    .
    signal all workers to start
    wait for all workers to finish
    do any clean up
}

Send_Service() {
    Put request in shared queue
    /*use a synchronization object for it*/
    Find a sleeping task.
    if sleeping task found
    {
        remove sleeping task from sleeping tasks list /*use synchronization */
        notify sleeping task /* Activate it */
    }
}

task1()
{
    wait for start
    forever
    {
        Check shared work queue
        if empty {
            put itself in the sleeping tasks queue
            wait for notification from Send_Service
        }
        fetch request /*use synchronization */
        perform task
    }
}

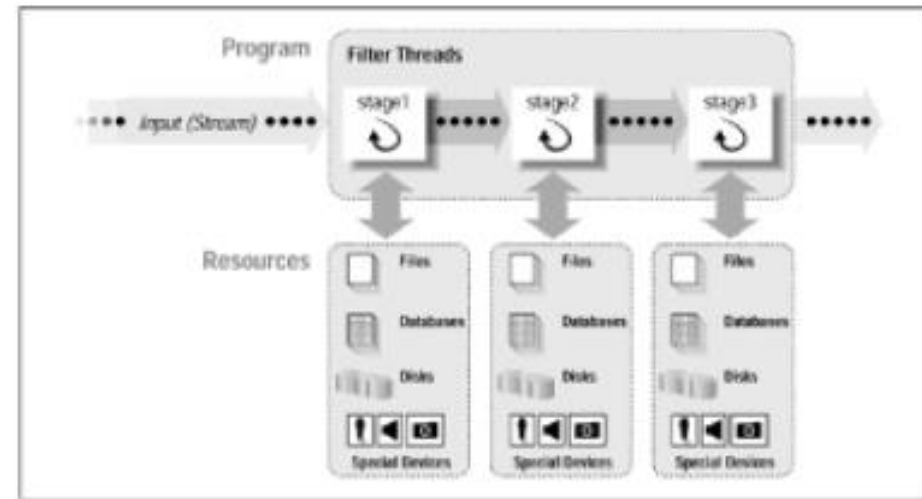
task2()
{
    tasks list /*use synchronization */
    The same as task1
}
    .
    .
    .

```

Pipeline Model

The pipeline model assumes:

- A long **stream of input**
- A **series of sub-operations** (known as **stages** or filters) through which every unit of input must be processed
- Each **processing stage can handle a different unit of input** at a time
- Fits well with the EZChip Network Processor design.



An automotive assembly line is a classic example of a pipeline. Each car goes through a series of stages on its way to the exit gates.

Instead of taking each car or instruction from start to finish before starting the next, a pipeline allows as many cars or instructions to be worked on at the same time as there are stages to process them. It still takes the same amount of time from start to finish for a specific car, but the overall throughput of the assembly line or computer chip is greatly increased.

Pipeline Model Example

```
creator()
{
    thread_create( ... stage1 )
    thread_create( ... stage2 )
    .
    wait for all pipeline threads to finish
    do any clean up
}

stage1()
{
    forever {
        get next queued input for the program
        do stage 1 processing of the input
        pass result to stage2 queue
    }
}

stage2()
{
    forever {
        get next queued input from previous
        thread in pipeline
        do stage 2 processing of the input
        pass result to next thread's queue in
        pipeline
    }
}
```

Inter process communication (IPC)

Inter process communication

Communication refers to the **transfer of information between multiple processes**.

- **synchronization is usually required**, as one process or another is going to have to wait for the communication to occur.
- Many IPC mechanisms may be used, They **vary in:**
 - **the amount of setup required**,
 - the **flexibility** of the service provided,
 - the **performance** capabilities (transaction rate and bandwidth),
 - **reliability**, and the **functionality provided**.
- Below is a list of IPC's:
 - Semaphores (binary and counting)
 - Mutexes and condition variables.
 - Message passing
 - Shared memory
 - Signals
 - Pipes and FIFOs.
 - Readers/writers locks

Inter process communication

categorizing all these mechanisms we get

Name	Reliable?	Flexible?	Fast?
Signals	Sometimes	Very	No
Messages	Yes	Not very	Not particularly
Semaphores	Yes	Very	Very
Shared Memory	Yes	Very	Very

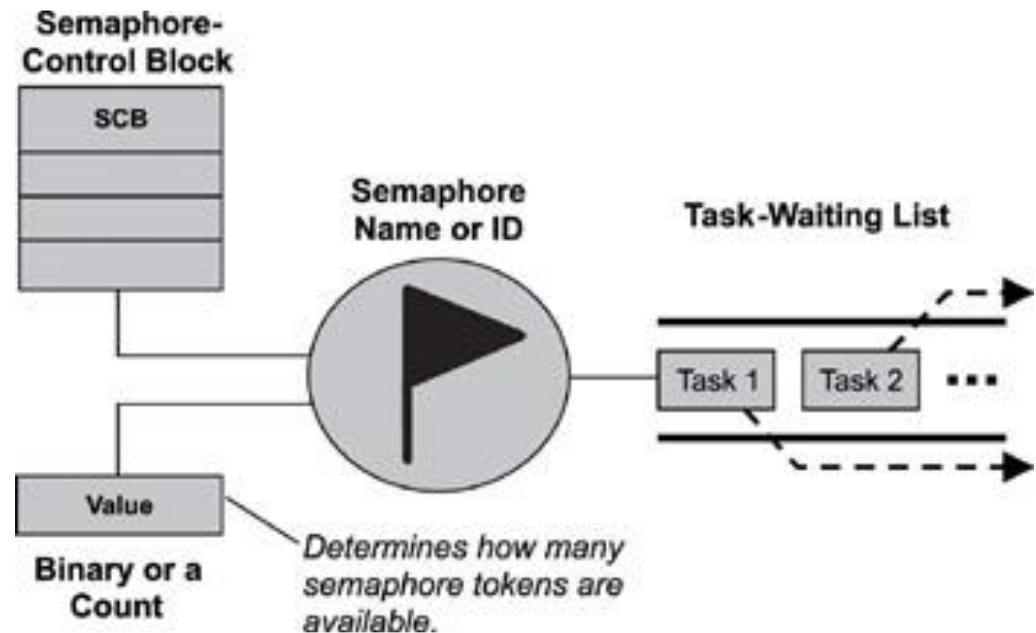
Semaphores

Defining Semaphores (1)

A semaphore (sometimes called a **semaphore token**) is a **kernel object** that one or more **threads of execution** can acquire or release for the purposes of **synchronization or mutual exclusion**.

When a semaphore is first created, the kernel assigns to it the following :objects

- an associated **Semaphore Control Block** (SCB),
 - a **unique ID**,
 - an **initial value** (binary or a count),
 - a **task-waiting list**
- A kernel can support many different types of semaphores, including **binary**, **counting**, and **mutual-exclusion** (mutex) semaphores



Defining Semaphores (2)

- A semaphore is like a **key** that **allows a task to carry out some operation or to access a resource**.
- If the task can **acquire the semaphore**, it can carry out the intended operation or access the resource.
- A single semaphore can be **acquired a finite number of times**.
 - In this sense, acquiring a semaphore is like acquiring the duplicate of a key from an apartment manager when the apartment manager runs out of duplicates, the manager can give out no more keys.
- when a **semaphore's limit is reached**, it can **no longer be acquired** until someone gives a key back or releases the semaphore.

Defining Semaphores (3)

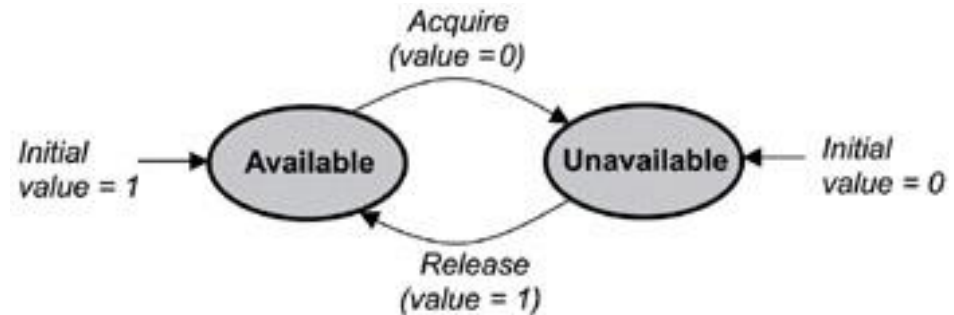
- The kernel tracks the number of times a semaphore has been acquired or released by maintaining a token count,
 - its initialized to a value when the semaphore is created.
 - task acquires a semaphore => token count is decremented;
 - task releases the semaphore => token count is incremented.
- If the token count reaches 0, the semaphore has no tokens left.
 - A requesting task, cannot acquire the semaphore, and the task blocks if it chooses to wait for the semaphore to become available.
 - task-waiting list - tracks all tasks blocked while waiting on an unavailable semaphore.
 - These blocked tasks are kept in the task-waiting list in either first in/first out (FIFO) order or highest priority first order.
- When an unavailable semaphore becomes available:
 - the kernel allows the first task in the task-waiting list to acquire it.
 - unblocked task is moved either to the running state, or the ready state.

Binary Semaphores

A binary semaphore can have a **value** of **either 0 or 1**.

When a binary semaphore's value is **0**, the semaphore is considered **unavailable** (or empty);

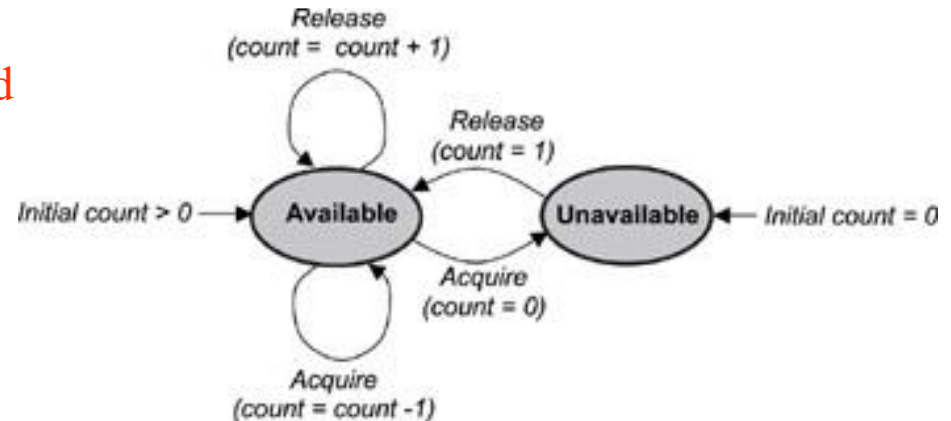
when the value is **1**, the **binary semaphore is considered available**



- when a binary semaphore is first created, **it can be initialized to either available or unavailable** (1 or 0, respectively).
- **Binary semaphores are treated as global resources**, which means they are shared among all tasks that need them.
- Making the semaphore a global resource **allows any task to release it, even if the task did not initially acquire it** (is it really a good thing?).

Counting Semaphores (1)

A counting semaphore **uses a count** to allow it **to be acquired or released multiple times**.



- When creating a counting semaphore, the semaphore is initialized with a count that denotes the number of semaphore tokens it has initially:
 - If the **initial count is 0**, the counting semaphore is created in the **unavailable state**.
 - If the count **is greater than 0**, the semaphore is created in the **available state**.

Counting Semaphores (2)

- One or more tasks can continue to acquire a token from the counting semaphore until no tokens are left.
- When all the tokens are gone, the count equals 0, and the counting semaphore moves from the available state to the unavailable state.
- To move from the unavailable state back to the available state, a semaphore token must be released by any task.
- As with binary semaphores, counting semaphores are global resources that can be shared by all tasks that need them.
- This feature allows any task to release a counting semaphore token.
 - Each release operation increments the count by one, even if the task making this call did not acquire a token in the first place.

Implementation of Counting Semaphore functionality.

The semaphore may be implemented using the following structure:

```
Struct Semaphore {  
    Max_Token_Count;  
    Current_Token_Count;  
    Queue_Of_Waiting_Tasks;    /* Use a list of TCBs */  
    Current_Owning_Task_Id;    /* May be used by mutexes only */  
    Reenterance_Count;        /* Used for by mutexes during recursive calls */  
}
```

Implementation of Counting Semaphore

In order to manage Counting Semaphores in VxWorks the following services have been provided:

SemCCreate() - create and initialize a counting semaphore

SemCTake() - give a semaphore – acquisition of a semaphore

SemCGive() - take a semaphore - relinquishing of a semaphore

```
SemCCreate (&Semaphore, MaxCount, InitialCount)
{
    Semaphore.Max_Token_Count = MaxCount
    if (InitialCount < 0)
        return with error
    Semaphore. Current_Token_Count = InitialCount
    return
}
```

```
SemCTake (&Semaphore)
{
    save and disable processor interrupts
    if (Semaphore .Current_Token_Count) > 0
    {
        dec_value_of(Semaphore. Current_Token_Count )
        restore and enable processor interrupts
    }
    else
    {
        remove CurrentTask from run queue
        mark CurrentTask as WaitOnSem
        add CurrentTask to Semaphore.Queue_Of_Waiting_Tasks
        restore and enable processor interrupts
        Reschedule
    }
}
```

Implementation of Counting Semaphore

Implementation of Counting Semaphore functionality – Counting Semaphores

SemCGive (&Semaphore)

```
{  
    save and disable processor interrupts  
    if (Semaphore. Queue_Of_Waiting_Tasks is not empty)  
    {  
        remove task p from Semaphore.Queue_Of_Waiting_Tasks  
        alter Task p status to runnable  
        add p to the run queue  
        restore and enable processor interrupts  
        Reschedule  
    }  
    else  
    {  
        inc (Semaphore. Current_Token_Count )  
        restore and enable processor interrupts  
    }  
  
    return  
}
```

Implementation Binary Semaphore.

```
SemBCreate (&Semaphore, InitialCount)  
{  
    Semaphore. Max_Token_Count = 1 /* Why */  
    if ((InitialCount < 0) or (InitialCount > 1))  
        return with error  
    Semaphore. Current_Owning_Count = InitialCount  
    return  
}
```

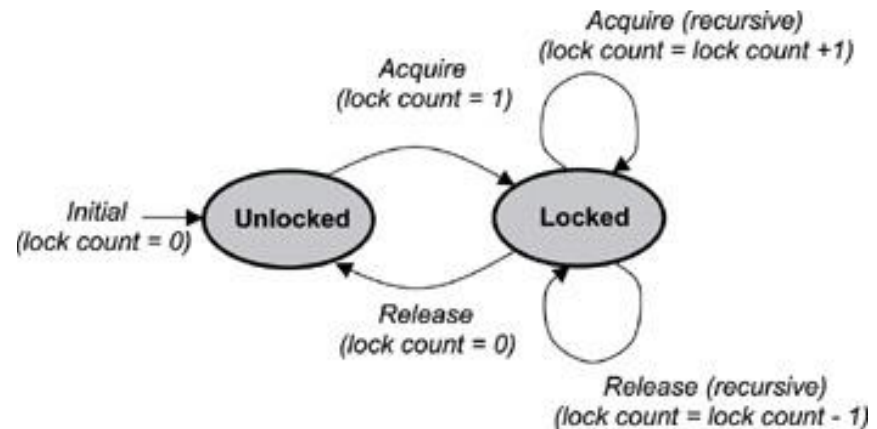
SemBTake and SemBGive are similar to SemCTake and SemCGive respectively

Why do we use caution with the definition of InitialCount?

Mutual Exclusion (Mutex) Semaphores

A mutual exclusion (mutex) semaphore is a **special binary semaphore** that supports

- **ownership,**
- **recursive locking,**
- **task deletion safety,**
- **priority inversion avoidance protocols**



- The states of a mutex are **unlocked** or **locked** (0 or 1).
- A mutex **is initially created in the unlocked state**, in which it can be **acquired by a task**. After being acquired, the mutex **moves to the locked state**.
- When the task **releases the mutex**, the mutex returns to the **unlocked state**.
- Mutexes support additional features not found in binary or counting semaphores

Mutex Implementation

```
SemMCreate (&Semaphore)  
{  
    Semaphore. Current_Token_Count = 1  
    return  
}
```

SemMTake is similar to SemBTake.

```
SemMGive (&Semaphore)  
{  
    Get ID of CurrentProcess  
    if (Semaphore. Current_Owning_Task_Id != ID)  
        return error  
  
    call SemBGive (&Semaphore)  
    return  
}
```

Mutex Ownership

Ownership of a mutex is gained when a task locks the mutex by acquiring it, a task loses ownership of the mutex when it unlocks it by releasing it.

When a task owns the mutex, it **is not possible for any other task to lock or unlock that mutex.**

Contrast this concept with the **binary semaphore**, which can be released by any task, even a task that did not originally acquire the semaphore.

Recursive Locking

Many mutex implementations also support recursive locking , which allows the task that owns the mutex to lock it multiple times in the locked state.

the kernel uses an internal lock count associated with the mutex to track the number of times that the task currently owning the mutex has recursively acquired it.

To properly unlock the mutex, it must be released the same number of times (What is the difference between counting semaphore???).

Task Deletion Safety

Some mutex implementations also have built-in task deletion safety.

Premature task deletion is avoided by using task deletion locks when a task locks and unlocks a mutex.

Enabling this capability within a mutex ensures that while a task owns the mutex, the task cannot be deleted.

Typically protection from premature deletion is enabled by setting the appropriate initialization options when creating the mutex.

Deadlock and Priority Inversion

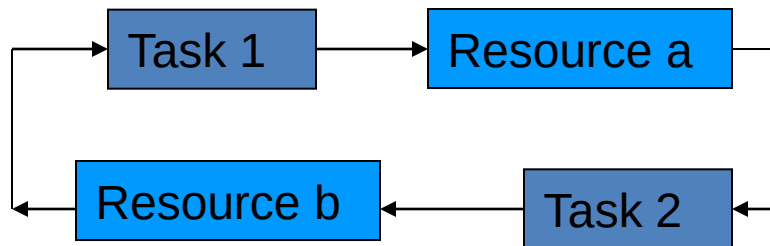
Mutexes are powerful tools for synchronizing access to shared resources. However, **they are not without their own dangers.**

Two of the most important problems to watch out for are:

- **Deadlock.**
- **Priority Inversion.**

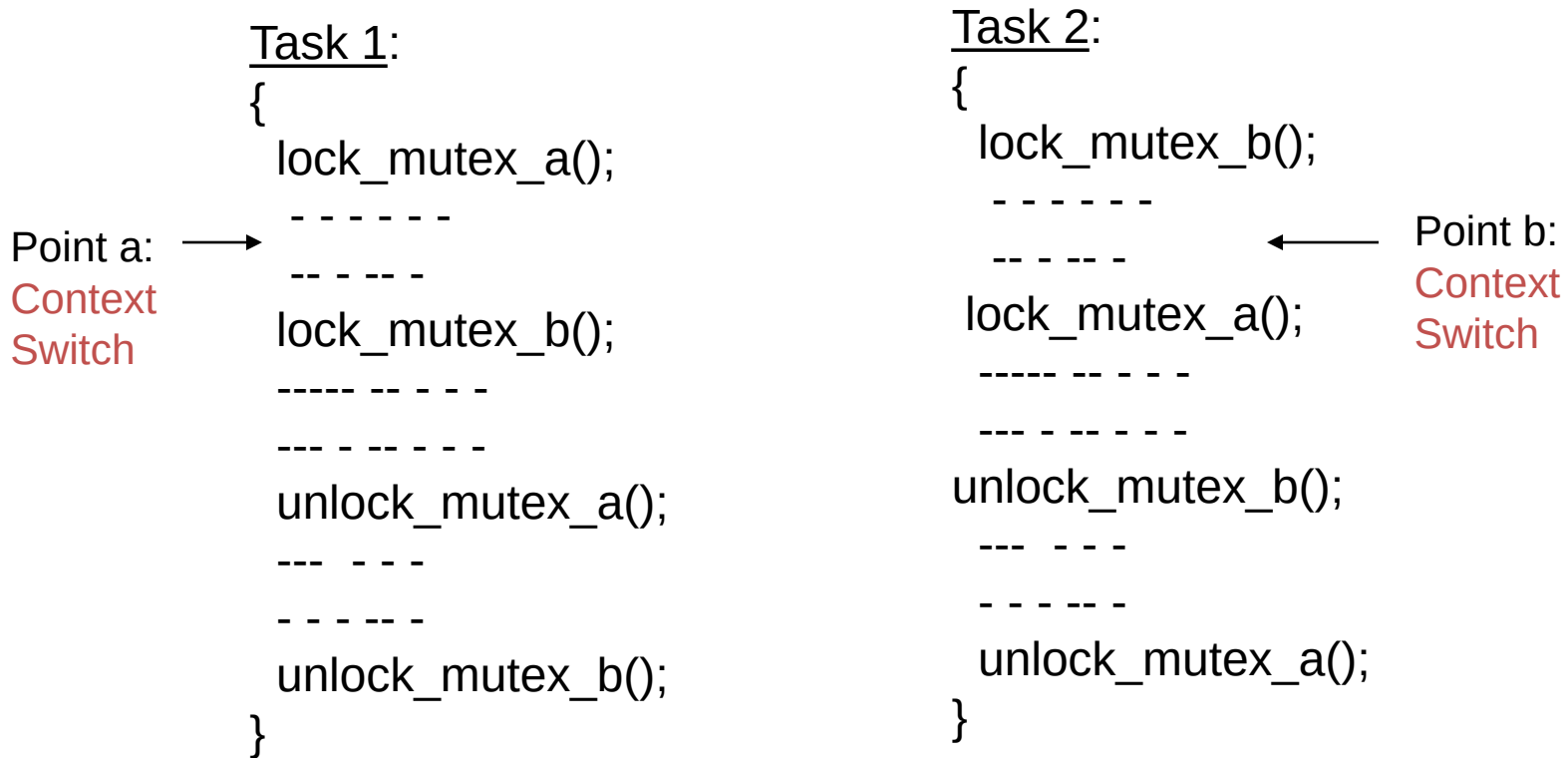
Deadlocks

- Deadlock can occur whenever there is a **circular dependency between tasks and resources**.
- The simplest example is of two tasks, each of which require two mutexes: A and B.
 - If one task takes mutex A and waits for mutex B
 - while the other takes mutex B and waits for mutex A,
- Both tasks are waiting for an event that will never occur.
This essentially **brings both tasks to a halt** and, though other tasks might continue to run for a while, **could bring the entire system to a standstill** eventually.
- The only way to end the deadlock is to reboot the entire system



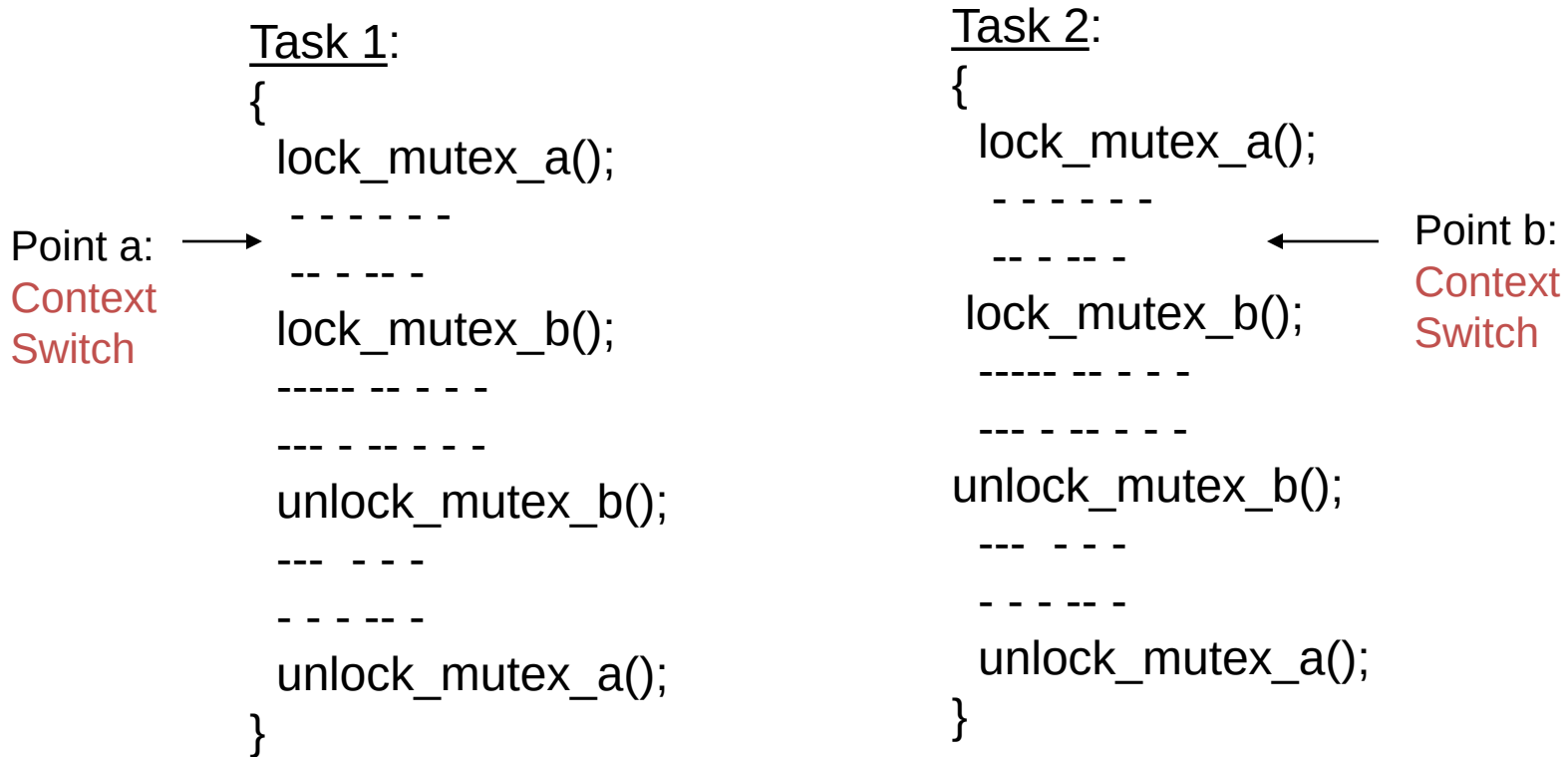
Task 1 wants Resource-a which is owned by Task2 which wants Resource-b which is owned by Task1

Deadlocks- how are they created?



What happens if task 1 starts and there is a context switch some where in point a

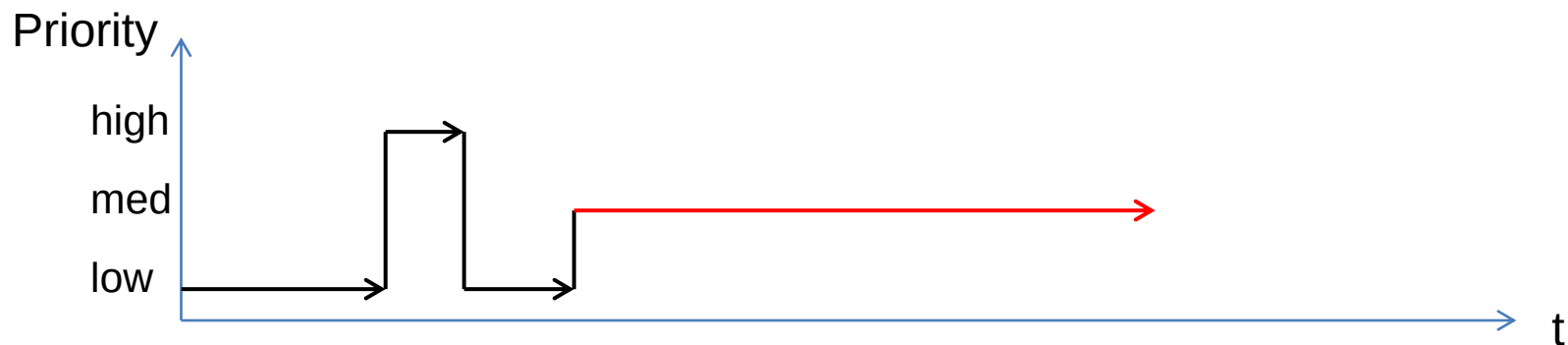
Deadlocks- how to avoid them



Solution: *Use the same order of locking in both (or all) tasks*

Priority Inversion

- Priority inversion commonly happens in poorly designed real-time embedded applications.
- Priority inversion occurs when a:
 1. **higher priority task is blocked and is waiting**
 2. **for a resource being used by a lower priority task,**
 3. **which has itself been preempted by an unrelated medium-priority task.**
- In this situation, the higher priority task's priority level has effectively been inverted to the lower priority task's level.



Priority Inversion Avoidance

Two common protocols are used for avoiding priority inversion include:

priority inheritance protocol :

- ensures that the **priority level of the lower priority task** that has acquired the mutex is **raised to that of the higher priority task** that has requested the mutex.
- when inversion happens, The **priority of the raised task is lowered to its original value after the task releases the mutex** that the higher priority task requires.

ceiling priority protocol :

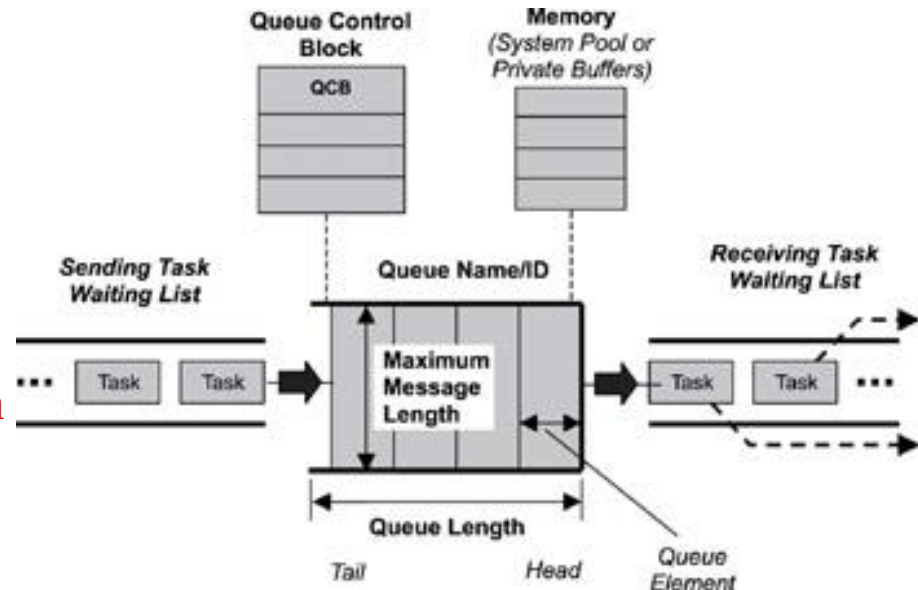
- ensures that the priority level of the task that acquires the mutex is automatically **set to the highest priority of all possible tasks** that might request that mutex when it is first acquired until it is released.
- When the mutex is released, the priority of the task is lowered to its original value.

Message Queues

Defining Message Queues

A message queue is a buffer-like object through which tasks and ISRs send and receive messages to communicate and synchronize with data.

A message queue is like a pipeline.
It temporarily holds messages from a sender until the intended receiver is ready to read them



message queue components:

- associated queue control block (QCB),
- message queue name,
- unique ID
- memory buffers,
- queue length and maximum message length,

Defining Message Queues

It is the **kernel's job** to assign a unique ID to a message queue and to create its QCB and **task-waiting list**.

- The developer supplies parameters such as:
 - **queue length**
 - **maximum message length**these determine **how much memory is required** for the message queue.
- The **message queue itself consists of a number of elements**, each of which can hold a single message.
- The elements holding the first and last messages are called the head and tail respectively.
- Some elements of the queue may be empty (not containing a message).
- The **total number of elements** (empty or not) in the queue **is the total length of the queue**.

Receiving \ Sending Task Waiting lists

A message queue has two associated task-waiting lists (these are lists of tasks waiting on the message queue).

Receiving task waiting list -

- consists of tasks that tried to read from the Message queue, but because it was empty, they had to wait on the queue (Blocked state).

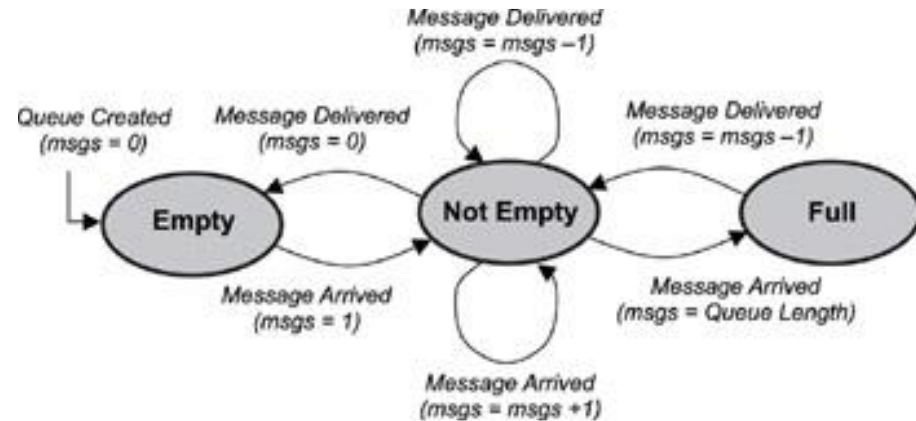
Sending task waiting list -

- consists of tasks that tried to write to the Message queue, but because it was full they had to wait on the queue (Blocked state).
- task-waiting list, is either
 - a **FIFO**
 - a **priority-based** order

Message Queue States

When a message queue is first created, its state is in the empty state.

- If a task attempts to receive messages from this message queue while the queue is empty, the task blocks
- if another task sends a message to the message queue, the message is delivered directly to the blocked task.
 - The blocked task is then removed from the task-waiting list and moved to either the ready or the running state.
- If another message is sent to the same message queue and no tasks are waiting in the message queue's task-waiting list, the message queue's state becomes not empty.



Full Message Queues

- As additional messages arrive at the queue, the queue eventually fills up until it has exhausted its free space.
- While a message queue is in this state (full), any task sending messages to it will not be successful unless some other task first requests a message from that queue, thus freeing a queue element.

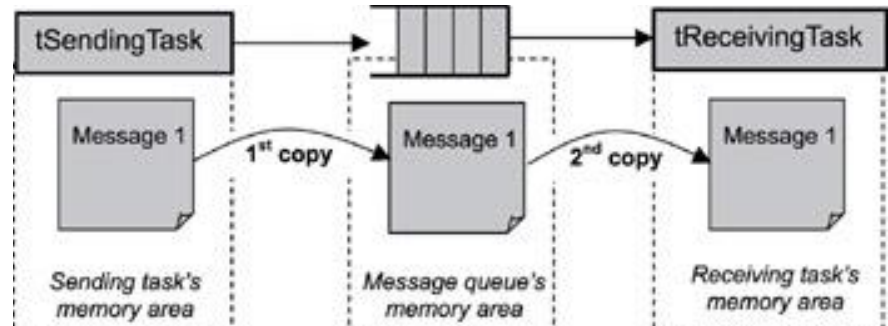
when a task attempts to send a message to a full message queue, Kernel implementations might behave differently:

- the sending function returns an error code to that task.
- The task status is changed to blocked, moving the blocked task into the sending task-waiting list, which is separate from the receiving task-waiting list.

Message Queue Content

- Some messages can be long and may exceed the maximum message length. One way to overcome the limit on message length is
 - Sending a pointer to the data, rather than the data itself.
 - This will improve both performance and memory utilization.
- When a task sends a message to another task, the message normally is copied twice:
 - The first time, when it is sent from the sending task's memory area to the message queue's memory area.
 - The second copy occurs when it is copied from the message queue's memory area to the receiving task's memory area.

copying data can be expensive, in a real-time embedded system we'll use pointers instead

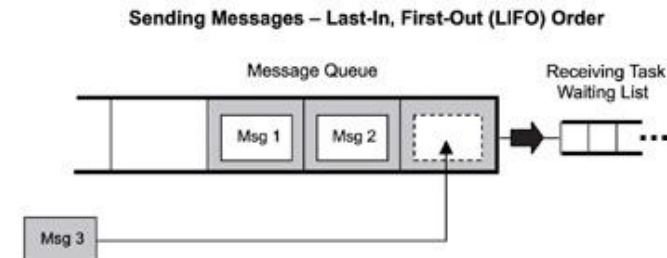
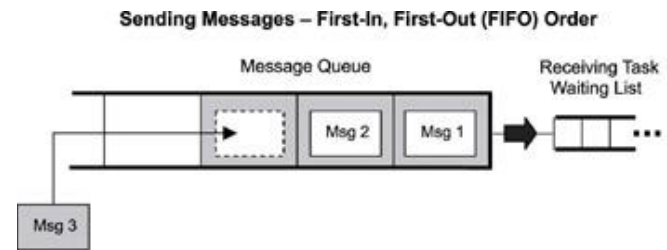


Creating and Deleting Message Queues

- When created, message queues are **treated as global objects and are not owned by any particular task.**
- When creating a message queue, **a developer needs to make some initial decisions:**
 - **message queue length**
 - **maximum size of the messages it can handle**
 - **the waiting order for tasks when they block on a message queue.**
- **Deleting a message queue automatically unblocks waiting tasks.**
 - The blocking call in each of these tasks returns with an error.
 - **Messages that were queued are lost when the queue is deleted.**

Sending Messages

- When sending messages, a kernel typically fills a message queue **from head to tail in FIFO** order.
- Some implementations allow **urgent messages to go straight to the head** of the queue.
- If all arriving messages are urgent, they all go to the head of the queue, and the queuing order effectively becomes last-in/first-out (LIFO).
- Many message-queue **implementations also allow ISRs to send messages** to a message queue.



Message Attributes

Sending and receiving messages Have the following attributes:

Non-Blocking (ISRs and tasks) –

- consist of **messages that must be sent without blocking the sender.**
- If a **message queue is already full, the send call returns with an error**, and the task or ISR making the call continues executing.
- **ISRs cannot block so this approach to sending messages is the only way for ISRs.**

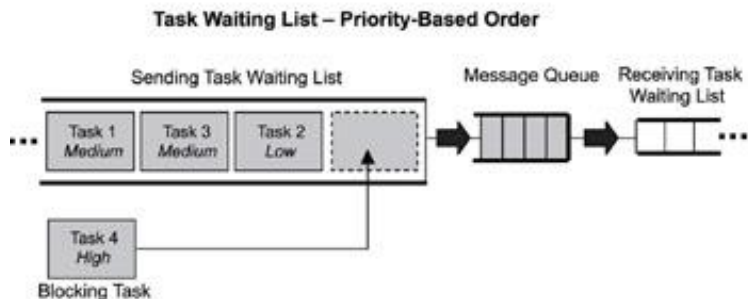
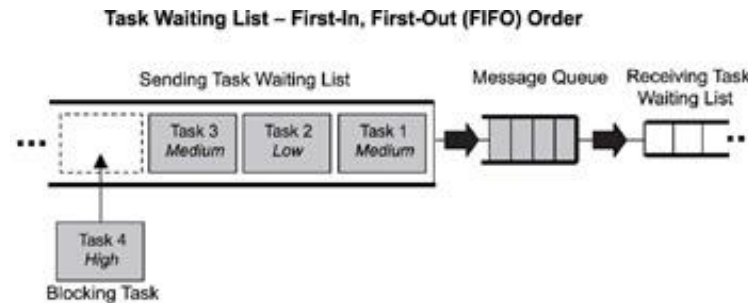
Blocking with a Timeout (tasks only) -

- Most times, the system should be designed so that a **task will block if it attempts to send a message to a queue that is full.**
- **The blocked task is placed in the message queue's task-waiting list (FIFO or priority-based order).**
- In the case of a **task set to block for a specified time**, the **task is unblocked if either a queue element becomes free or the timeout expires**, in which case an error is returned.

Message Sending types

Blocking Forever (tasks only) –

- In the case of a **task set to block forever** when **sending a message**, the task blocks **until a message queue element becomes free** (e.g., a receiving task takes a message out of the queue).



Receiving Messages

- Tasks can receive messages with different blocking policies the same way as they send them with:
 - a policy of not blocking,
 - blocking with a timeout,
 - or blocking forever.
- For the **message queue to become full**, the rate at which messages are posted in the message queue must be greater than the rate at which messages are removed.
- Only when the message queue is full does the task-waiting list for sending tasks start to fill.
- Conversely, for the task-waiting list for receiving tasks to start to fill, the message queue must be empty.

Reading Received Messages

Messages can be read from the head of a message queue in two different ways:

Destructive Reading –

- when a task successfully receives a message from a queue, the task permanently removes the message from the message queue's storage buffer

Non-Destructive Reading –

- a receiving task peeks at the message at the head of the queue without removing it usually kernel implementations don't support it.

Exceptions and Interrupts

What are Exceptions?

- An exception is **any event that disrupts the normal execution of the processor** and **forces the processor into execution of special instructions in a privileged state.**
- An **interrupt**, sometimes called an **external interrupt**, is an asynchronous exception triggered by an event that an external hardware device generates
- Exceptions can be classified into two categories:
 - **Synchronous:**
 - Interrupt caused by **internal events**, such as events generated by the execution of processor **instructions**.
 - for example: An arithmetic operation that results in a **division by zero raises a exception.**
 - **Asynchronous:**
 - Interrupt caused by **external events**, which are associated with **hardware signals**.
 - The sources of these hardware signals are external hardware devices.

Usage of exceptions

Exceptions and interrupts **provide a facility for embedded hardware** (either internal or external to the processor) to **gain the attention of the CPU**.

Interrupts are a **means of communicating** between the **hardware and the application currently running on an embedded processor**.

Exceptions and interrupts help the embedded engineer in three areas:

- **Detecting internal errors and special conditions.**
- **Concurrency of hardware operation while CPU is busy.**
 - **Incoming Ethernet packet.**
 - **Disk IO completion.**
 - **Timer expiration.**
 - **DMA completion.**
- **Debugging.**

Reset vector and triggers for exception handling.



RT Embedded
Linux Solutions

- Handling IRQ (and FIQ only in ARM Architecture) is triggered by external interrupts.
- Handling SWI command is triggered by issuing a system call or an assembly SWI command by the user program. This is a “[Software Interrupt](#)”.
- Signals are implemented via Software Interrupts as well.
- Handling all other entries are triggered by the internal CPU hardware while it detects an illegal flow, such as:
 - [Divide by Zero.](#)
 - [Access to unavailable memory.](#)
 - [Issuing illegal assembly command.](#)

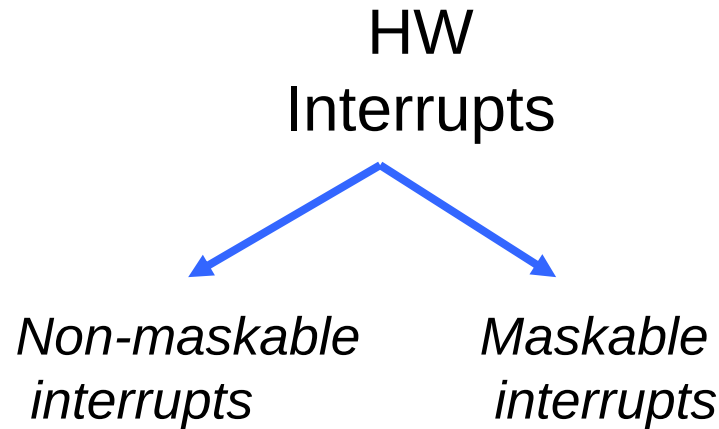
Handling exceptions

Exceptions and the Interrupt vector are tightly connected.

Whenever an exceptions occurs – the CPU performs the following tasks:

1. Switch to privilege mode which allows it to access resources which are not available to user programs.
2. Jumps to the Interrupt vector entry that is dedicated for the specific exceptions.
3. Jumps from this specific entry to an assembly routine.
4. The assembly routine saves all necessary registers in the stack.
5. The assembly routine issues a call to a C function which issues the handling on behalf of the OS (ISR) – including Re-Scheduling if needed.
6. Context of the program that is aligned to run are restored.
7. Exceptions returns and mode is restored.
8. Operation proceeds according to 6.

Types of Interrupt



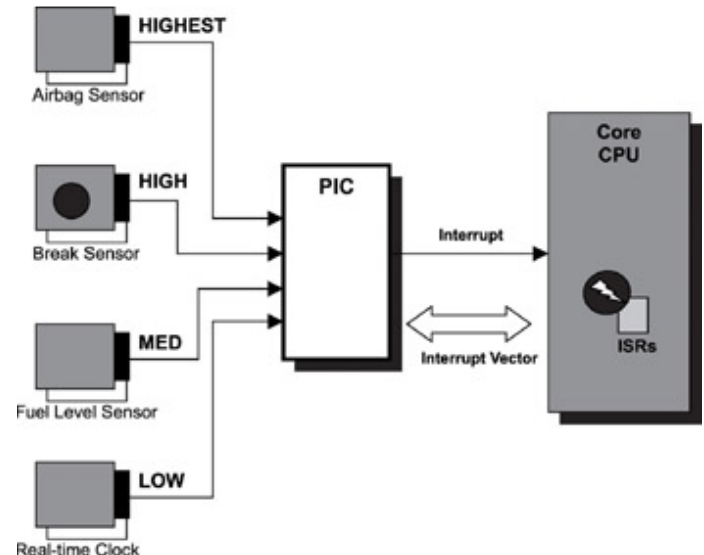
Maskable interrupts - ignored by the control unit as long as it remains masked.

Nonmaskable interrupts - are always recognized by the CPU, critical events (such as hardware failures).

Programmable Interrupt Controllers

Most embedded designs have many devices, these multiple external interrupt sources are **prioritized by a PIC device**.

The PIC Prioritizes multiple interrupt sources so that at any time **the highest priority interrupt is presented to the core CPU for processing**.



The PIC has a set of interrupt request lines.

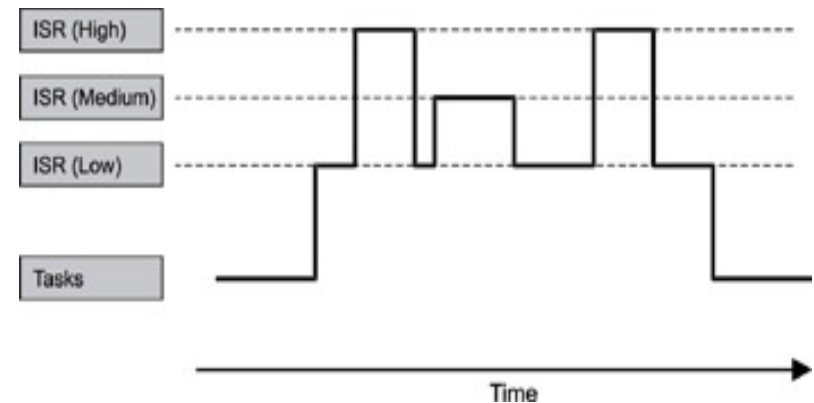
An external source generates interrupts by asserting a physical signal on the interrupt request line.

Each **interrupt request line has a priority assigned to it**, and connects to one distinct interrupt request line.

Nested Interrupts

Nested exceptions refer to the ability for **higher priority interrupts to preempt the processing of lower priority ones.**

When the higher priority service routine is complete, the earlier running service routine returns to execution.



Example:

- A task is executing.
- A low-priority interrupt then becomes active, and the associated service routine comes into context.
- While this service routine is running, a high-priority interrupt becomes active, and the lower priority service routine is preempted.
- The high-priority service routine runs to completion, and control returns to the low-priority service routine.

Special care with nested exceptions



RT Embedded
Linux Solutions

There are a few rules that should be followed while designing exceptions handlers with respect to nested calls:

1. Each exception type – including interrupts - is associated with its own stack.
2. Since nesting of some exceptions (such as interrupts) can not be always predicted – registers that may be modified, must be pushed to this stack. Otherwise, nesting is impossible.
 - The scheme that was described in the previous slide will not work unless taking nested call considerations.
3. Recursive calls from within exceptions handler or defining large array variables are prohibited, since the stack may be exhausted.

Interrupts and tasks

Privileged tasks can issue the following commands to control interrupts:

- IntEnableAll
- IntDisableAll – Not effective for NMI
- IntEnableSpecific
- IntDisableSpecific – Not effective for NMI

Disabling interrupts is the most effective (and devastating) way to prevent task preemption.

Disabling interrupts for short time in one-CPU system is one of the ways to prevent race conditions in case both the ISR and the task may have access to the same global variable/resource.

However, disabling interrupts for long time may hurt system performance and integrity.



**RT Embedded
Linux Solutions**

Signals

What is a signal ?

A signal is a **software interrupt** that is generated when an event has occurred. It diverts the signal receiver from its normal execution path and triggers the associated asynchronous processing.

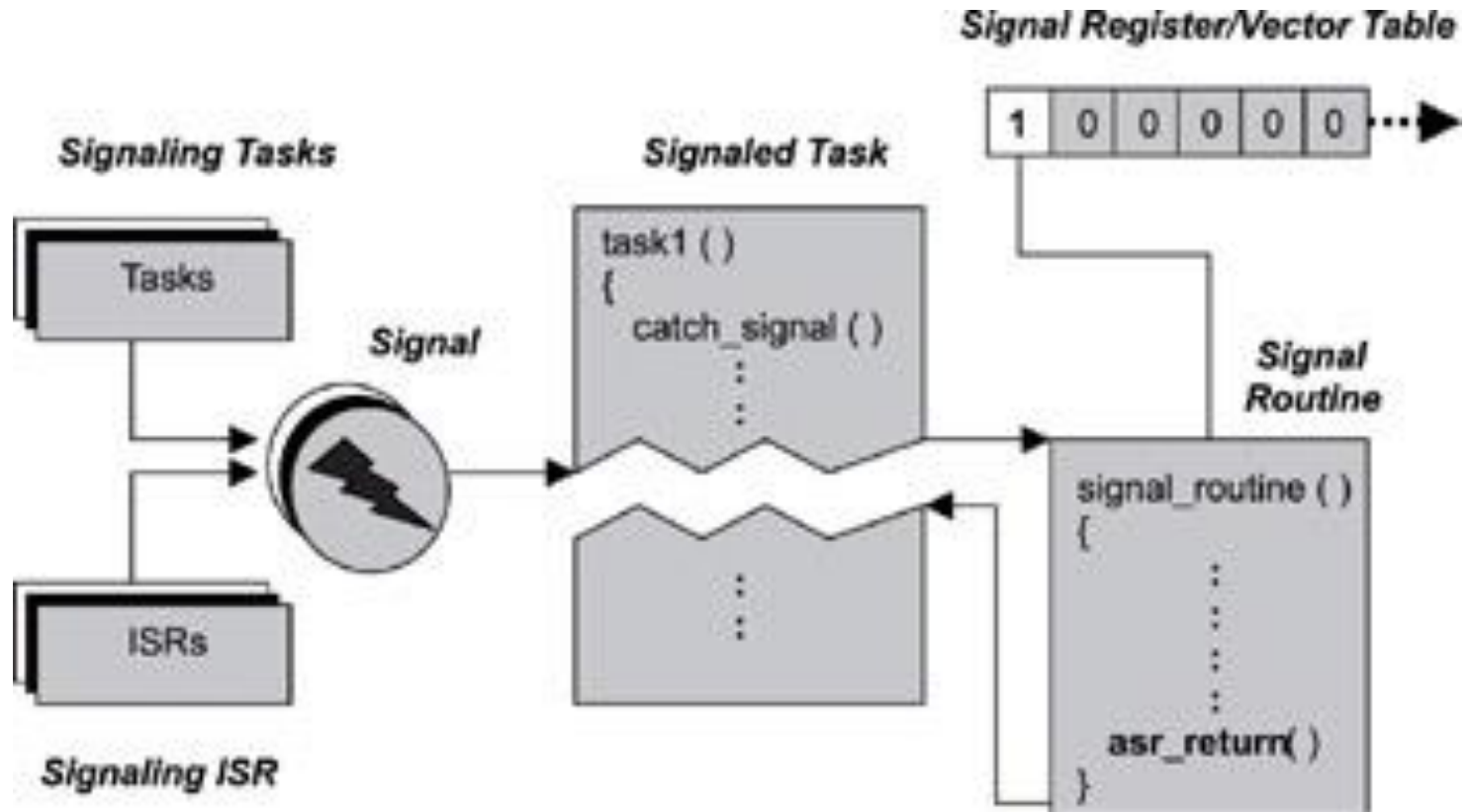
Essentially, signals notify tasks of events that occurred during the execution of other tasks or ISRs (Signals can be issued by both).

Each signal is issued with two parameters:

- PID of receiving task.
- Signal ID – location in the “**Signal Vector**”.

By default-tasks have no internal handling mechanism of received signals. However – they can set such a mechanism. This is the “**Signal Routine**”.

Signal flow.



Signal handling by the receiver.

The default response by a receiving task is to terminate immediately.

However, the programmer may decide to set the appropriate entry in the vector table to point either to a system function or to his own Signal Handler Routine.

The following values are valid:

IGNORE – Instructing the OS to disregard signals dedicated to this task. In this case signals of certain type will not be handled by the OS default routine, and the task will not be interrupted and not terminated.

DEFAULT – Instruct the OS to perform its default handling. The programmer may want to use this value if the vector table entry was previously set to some other value.

• **A function address** – The address of the Signal Handler Routine that was written by the programmer.

Signals in Linux: Kill system call

One process can send a signal to another process by using the kill system call.

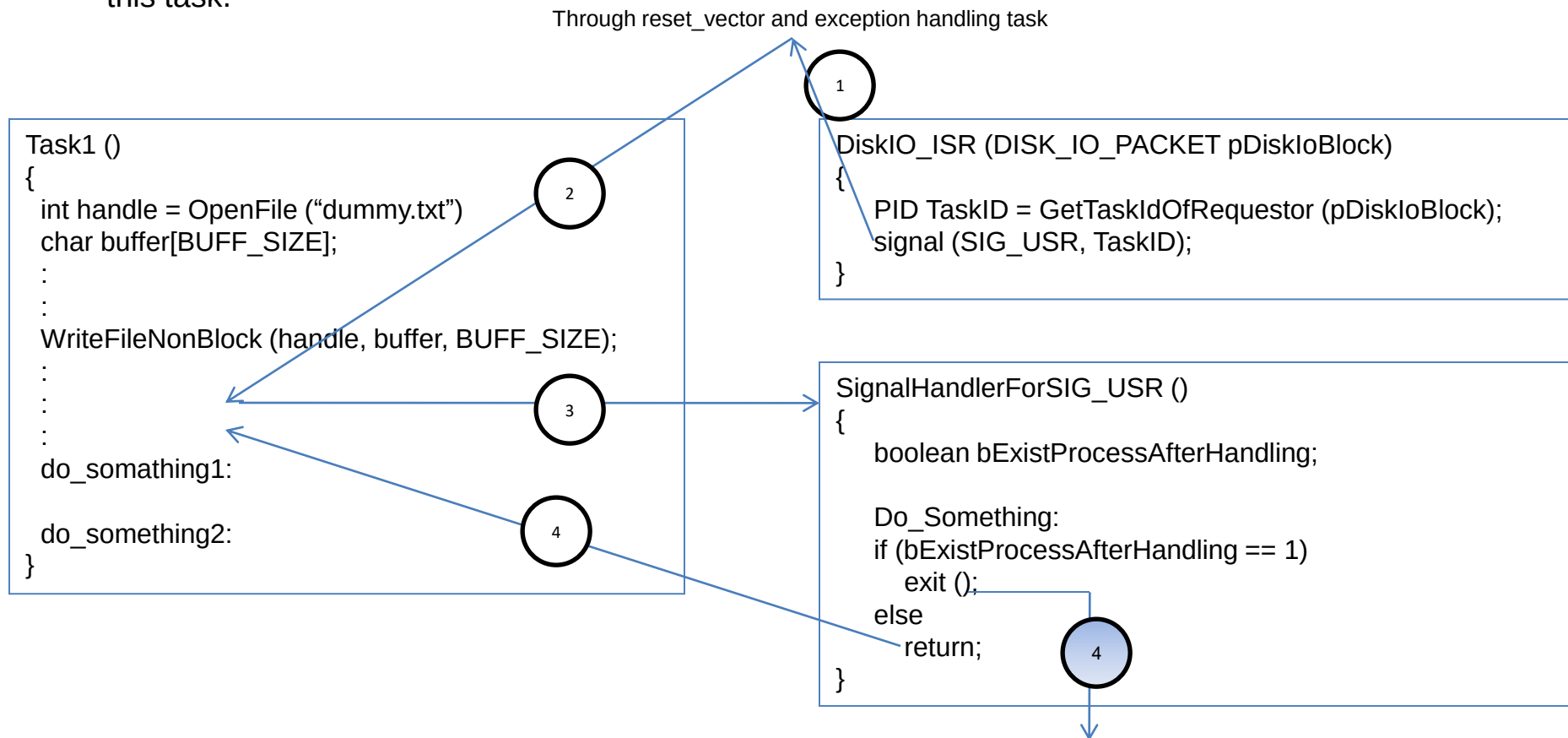
To send a signal, the sender has to know is the process ID of the victim. There are also some permission checks to be done.

You can send a signal to another process if your process has the same real or effective user ID as the victim process (usually this means the user who created the sender process also needs to create the victim process)

kill is so named because the result of this action is usually the abrupt and abnormal termination of the target process, unless the other process has done something special, like set up a handler, in preparation for the signal (SIGUSRI \ SIGUSR2).

Generating a signal - Examples

Example 1: Disk IO operation issued by Task1 was completed during execution of this task.



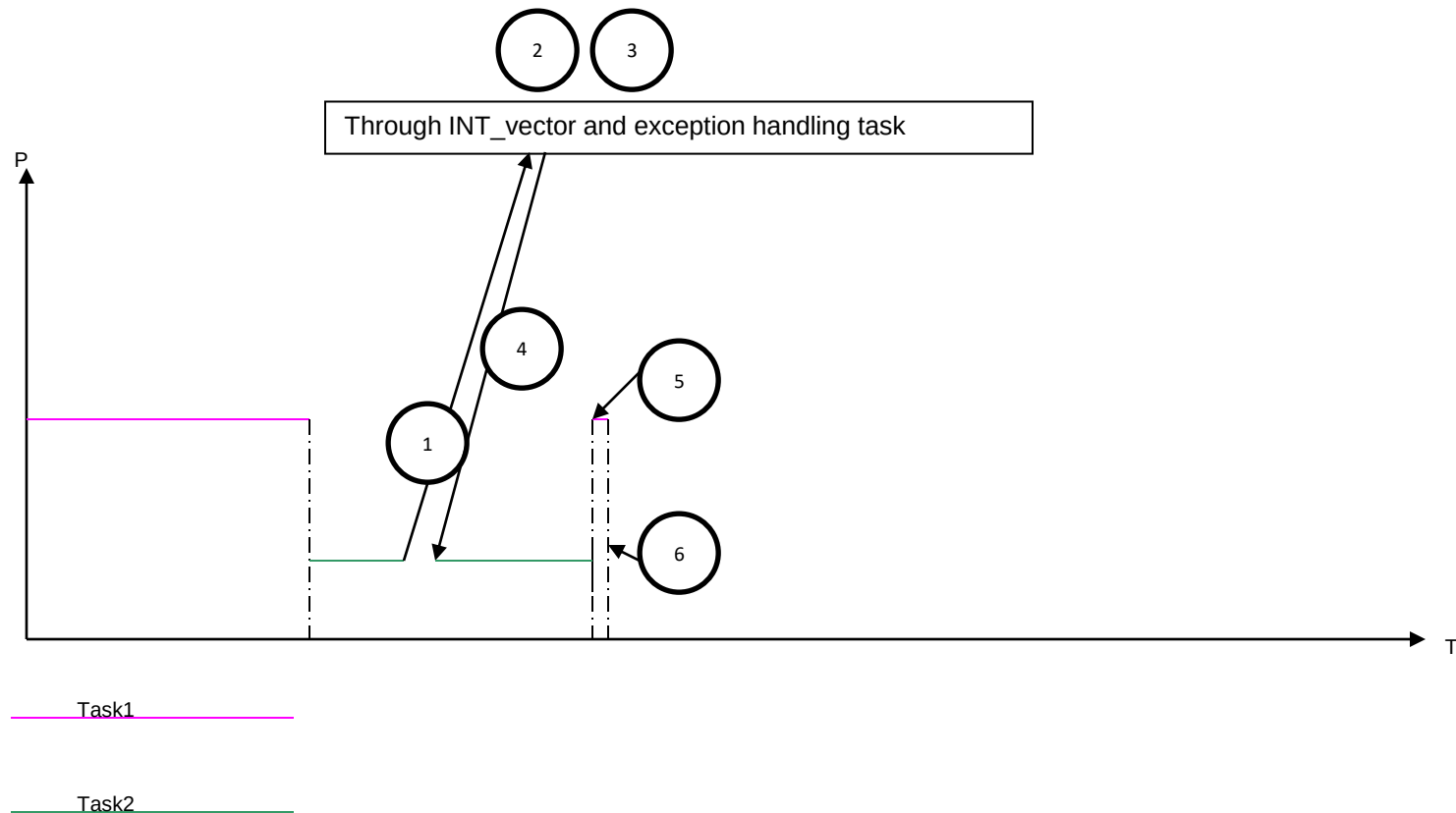
Generating a signal - Examples

Example 2: Issuing a signal to terminate a process using kill command .

- Task1 and Task2 co-exist in the system with PIDs 0x1098 and 0x1099 respectively. Task1 is in “ready” state while Task2 is running. Task2 sends the command *“kill -9 0x1098”* in order to terminate Task1.
 1. The OS will issue a software interrupt with the corresponding parameters: signal_code and PID of the receiving task.
 2. The processor jumps to its INT_vector to the entry representing software_interrupt handling.
 3. The software interrupt handler routine sends a message or wakes up a blocking semaphore acquired by the exception handling system task.
 4. **The exception handling system task locates the receiving process and puts a message in the task’s system message queue and issues the Scheduler by calling reSchedule.**
 5. Once Task1 is allowed to run – its signal handler runs first.
 6. If signal handler of this task was not re-adjusted by the programmer – Task1 is terminated immediately.

Generating a signal - Examples

Example 2: Issuing a signal to terminate a process using kill command - Continue



Should you use signals for IPC?

Because **a process can send a signal to another process**, signals can be used for inter-process communication.

Signals **are almost never the appropriate choice as a communication mechanism**;

1. They're too slow.
2. They are too limited .
 - The receiver can use only the signal ID and one associated 32 bit value to decide how to deal with it.
3. In some old implementations - queuing is insufficient or just does not exist.
4. Signals asynchronously interrupt your process in ways that require clumsy coding to deal with.

What Should signals be used for?

Signals are mostly used for

1. Timer expiration
2. Asynchronous I/O completion
3. Notifying an exception.
4. Abnormal termination of a process.

Signals offer an **advantage that other, higher-performance communication mechanisms do not support: signals are asynchronous**. That is, a signal can be delivered to you while you are doing something else.

Identifying and Prioritizing of Signals

Signals are identified and prioritized by their number.
Higher signal numbers mean lower priority.

The numbers use symbolic names and are defined in the header `<signal.h>`.
`/usr/include/asm/signal.h`

Each number has a particular meaning and effect on the process that gets the signal.

two signals are available for your application to use:

- **SIGUSR1**
- **SIGUSR2.**

Note

Never use the actual number of the signal; you never send signal 13 to a process.
That's because 13 means something on one system, but may mean something else entirely on another system. use symbolic names Instead.

Signals Required by POSIX

Signal Name	Used For
SIGABRT	Abnormal termination, abort
SIGALRM	Alarm clock expired (real-time clocks)
SIGFPE	Floating point exception
SIGHUP	Controlling terminal hung up (Probably a modem or network connection)
SIGILL	Illegal instruction exception
SIGINT	Interactive termination (usually CTRL-C at the keyboard)
SIGKILL	Unstoppable termination (signal 9 on most UNIX systems)
SIGSEGV	Memory access exception
SIGPIPE	Writing to a pipe with no readers
SIGQUIT	Abnormal termination signal (interactive processes)
SIGTERM	Terminate process
SIGUSR2	Application-defined uses
SIGUSR1	Application-defined uses

FreeRTOS Overview

FreeRTOS Overview

- FreeRTOS can be thought of as a Scheduler + library that provides multi-tasking capabilities to what would otherwise be a bare metal application.
- FreeRTOS can be built with (approximately) twenty different compilers, and can run on more than thirty different processor architectures.
- Each supported combination of compiler and processor is considered to be a separate **FreeRTOS port**.

FreeRTOSConfig.h

- FreeRTOS is configured by a header file called FreeRTOSConfig.h.
- Used to tailor FreeRTOS for use in a specific application.
- FreeRTOSConfig.h contains constants such as
 - configUSE_PREEMPTION - defines whether the co-operative or pre-emptive scheduling algorithm will be used
- FreeRTOSConfig.h should be located in a directory that is part of the application being built.
- No need to create the FreeRTOSConfig.h from scratch, its provided with every demo application per architecture.
- Adapt it t your application needs.
- Link to <https://www.freertos.org/a00110.html> – elaborate regarding each const

FreeRTOS Distribution

- FreeRTOS is distributed in a single zip file.
- The zip file contains:
 - ❑ source code for all the FreeRTOS ports,
 - ❑ project files for all the FreeRTOS demo applications.
 - ❑ a selection of FreeRTOS+ ecosystem components
 - ❑ a selection of FreeRTOS+ ecosystem demo applications.

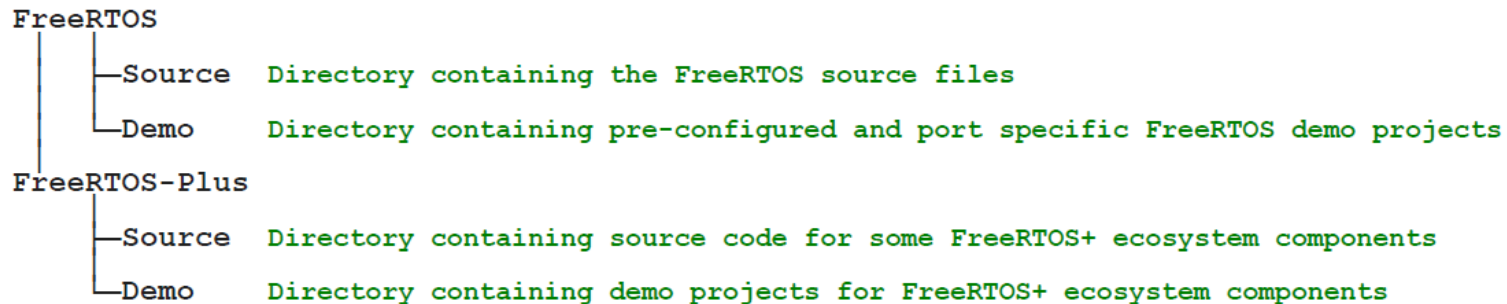


Figure 1. Top level directories within the FreeRTOS distribution

FreeRTOS Source Files Common to All Ports



RT Embedded
Linux Solutions

- Common FreeRTOS source code are located directly in the FreeRTOS/Source directory, as shown in Figure 2.

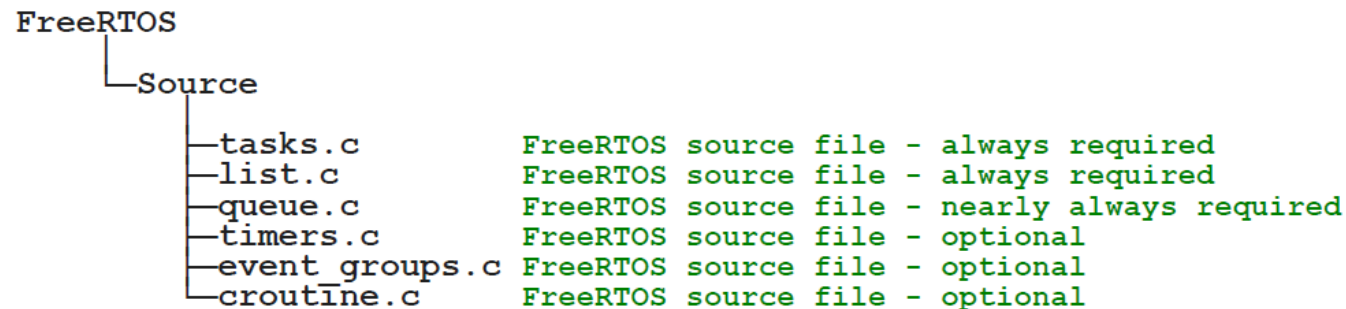


Figure 2. Core FreeRTOS source files within the FreeRTOS directory tree



**RT Embedded
Linux Solutions**

Appendix

Simple Programmable Logic Devices (SPLDs)

What is an PLD?

- Programmable Logic Devices (PLDs): are chips with a large number of gates and flip flops that can be configured with software to perform a specific logic function or perform the logic for a complex circuit. Major types of PLDs are:
 - Simple Programmable Logic Devices (SPLDs):
 - Complex Programmable Logic Devices (CPLDs):
 - Field Programmable Gate Array (FPGA):

What is an PLD?

- Advantages of PLDs over fixed-function chips include:
 - Reduced complexity of circuit boards
 - Lower power requirements
 - Less board space
 - Simpler testing procedures
 - Higher reliability
 - Design flexibility

Field Programmable Gate Arrays (FPGA)

What is an FPGA?

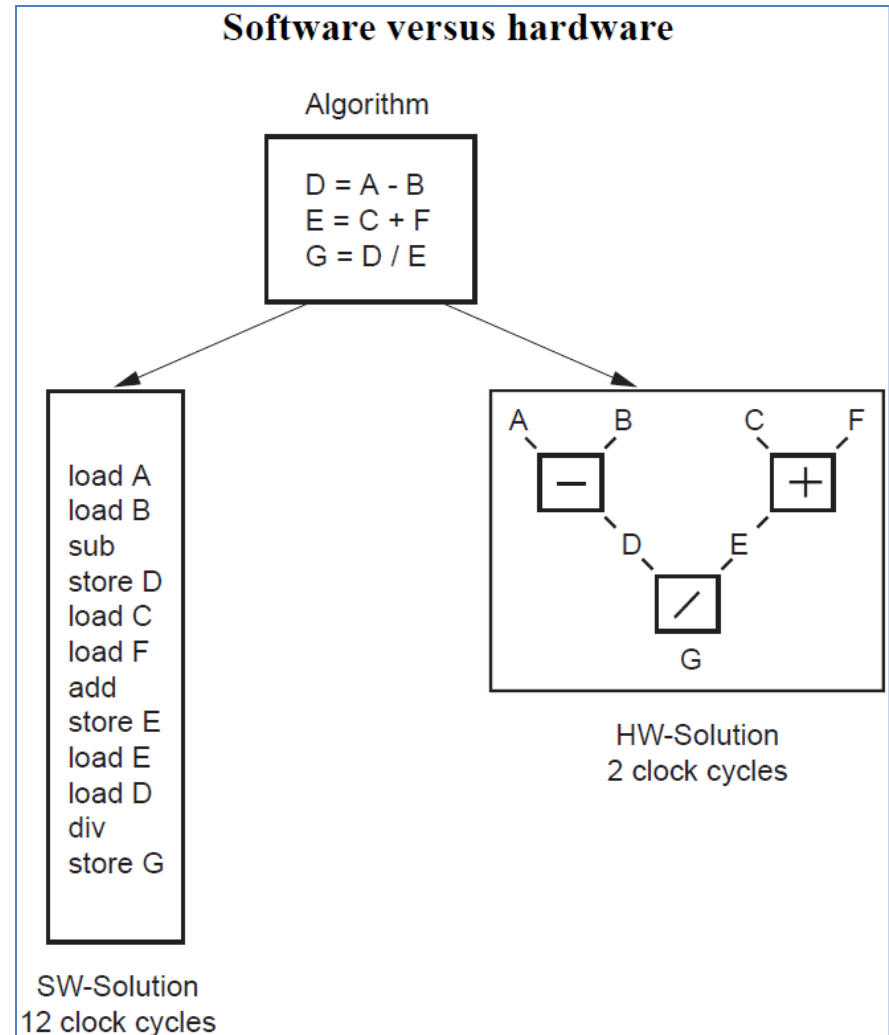
- It is primarily a semiconductor device that can be configured by the designer.
- The term "field-programmable" means the device is programmed by the customer, not the manufacturer.
- Can be programmed using a logic circuit diagram or source code in VHDL or Verilog
- It offers partial re-configuration of a portion of design
- An **FPGA (Field Programmable Gate Array)** is a reprogrammable chip which contains hundreds of thousands of logic gates that internally connects together to build complex digital circuitry

Benefits of FPGA's

- Real-time analysis of high-rate data streams (Performance)
- Deterministic hardware dedicated to every task (Reliability)
- Nonrecurring engineering expenses (Re-configurability)
- Radiation Hardened and Program Integrity. (Durability)
- Flexible and rapid prototyping (Development)

FPGA Performance

- FPGAs excel at computing **non-data** dependent algorithms **in parallel**.
- **Customizable data path** and ALU allow very large amounts of data to be transferred and computed within several clock cycles.
- Despite **lower clock frequencies**, FPGA's can outperform conventional CPU's on certain data processing tasks

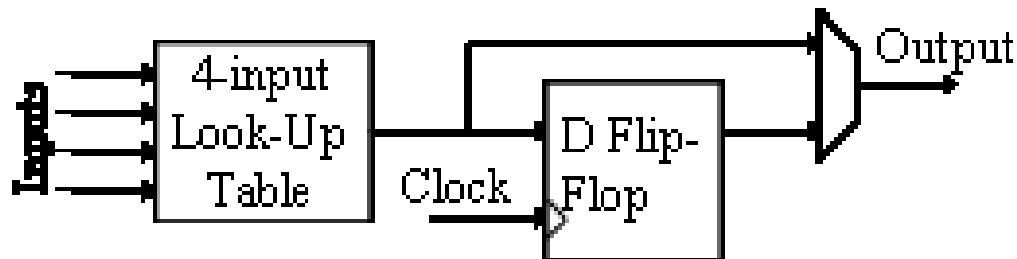


Disadvantages of FPGAs over ASICs:

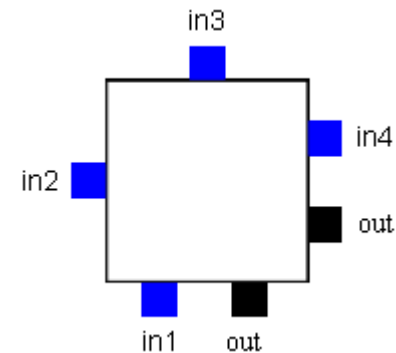
- The reasons they may NOT fit your design are:
 - **Power consumption** - FPGAs fundamentally use a lot more power than ASICs
 - **Price** - they also fundamentally cost more
 - **Speed** - ASICs can still blow any FPGA away in speed although design techniques can help with this issue
 - **Density** - ASICs can still pack a lot more logic into a single chip than an FPGA
 - **IP** - modern, complex IP (a complete PCI Express or Hyper-transport core for example) may take up most or all of an FPGA but only 10% of an ASIC

Architectural Features of FPGAs

- Common FPGA architecture involves:
 - Configurable Logic Blocks (CLBs)
 - I/O pads
 - Routing Paths usually of the same width (# of wires)

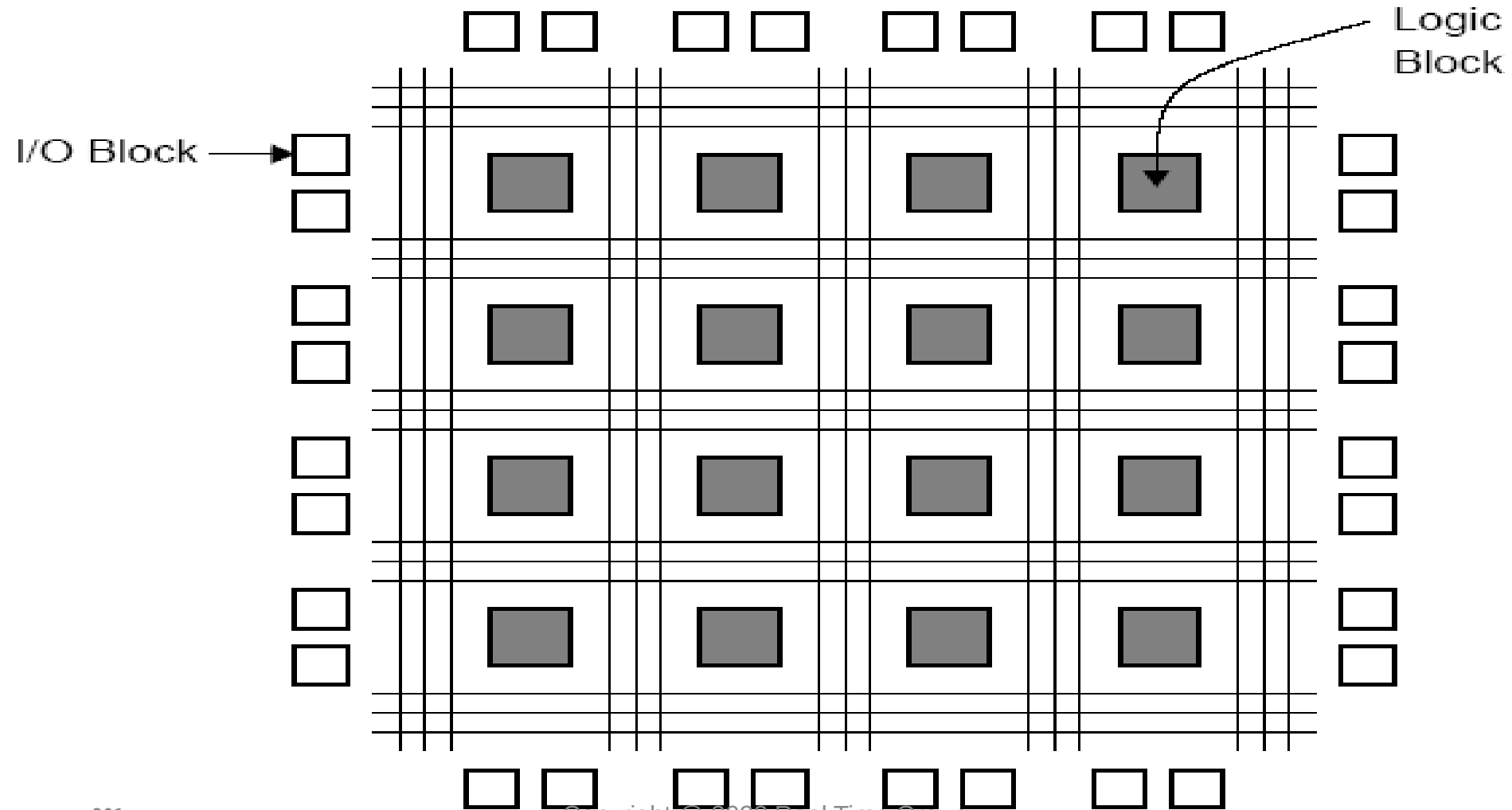


***Standard Logic
Block***



***Logic Block
Pin Assignment***

Architectural Features of FPGAs



Special attributes to

- ARM team for its Embedded Software Training
- **Analog Devices Embedded workshop**
- Atmel University
- NXP Training Content



**RT Embedded
Linux Solutions**

END

Hope you enjoyed

Presented By

Benny Cohen

rt-group.co.il

List of our Related Courses



**RT Embedded
Linux Solutions**

Software Programming Courses

- C for Embedded Systems
- C++ Programming
- C++ for Embedded Systems
- C++ Design Patterns
- Programming Java Applications
- Programming Python Applications \ Scripts

Embedded – Bare Board Courses

- RT & Embedded Concepts
- Embedded (ARM) Programming
- ARM Cortex A
- ARM Cortex R
- ARM Cortex M

Linux Courses

- Linux Fundamentals
- Linux Kernel & Device Drivers
- Linux System\ User Space Programming
- Embedded Linux Programming
- RT Embedded Linux Development